

ACTIVATION SPARSIFICATION IN TRANSFORMERS: PRESSURE, THRESHOLDING, AND SITE PLACEMENT

Anonymous authors

Paper under double-blind review

ABSTRACT

Useful activation sparsity depends on how zeros are induced, which operations they affect, and whether exploiting them repays execution overhead. We study pressure, thresholding and site placement through 54 matched Pythia pretraining conditions on MiniPile: detailed comparisons at 14M and selected complete recipes at 70M and 410M. Model-wide activation sparsity, $\mathcal{S}_{\text{model}}$, counts zero-operand scalar multiplications in a declared workload; architectural reach, R_{arch} , describes the portion structurally affected by the selected sites. Pressure’s marginal quality–sparsity effect changes with the threshold and target objective, from improving both observed axes to exchanging quality for additional zeros. Distribution measurements and operation counts also show that the seven-site recipe can produce greater model-wide sparsity despite fewer pooled FFN zeros, because its attention contribution outweighs reduced projection contributions. The high-threshold complete-recipe advantage persists across the three sizes, but lower-threshold orderings vary. On one RTX5090, a selected implementation achieves a $1.234\times$ geometric mean speedup over native execution across the 30 14M checkpoints for BF16, uncached 2,048-token inference. Sparse paths add $1.043\times$ over matched fusion, helping 14 checkpoints; attention skipping slows every checkpoint. These single-seed comparisons motivate evaluating an intervention’s paired quality cost, affected operations and execution granularity together.

1 INTRODUCTION

Activation sparsity can reduce inference cost when specialized kernels exploit zero activations (Liu et al., 2024; Cetin et al., 2026). Choosing how to create those zeros requires more than a local sparsity percentage: zeros at different sites affect different amounts of multiplication work, and skipping that work can itself add overhead. We ask how pressure, thresholding and their placement change the quality cost of sparsification, and which of the resulting opportunities translate into faster execution.

Existing recipes combine these choices in different ways. TEAL applies post-hoc magnitude thresholding with allocation across projections; ProSparse combines ReLU, progressive activation pressure and threshold shifting; and Q-Sparse uses top- k selection at attention and FFN projection inputs (Liu et al., 2024; Song et al., 2024; Wang et al., 2024). Their ablations motivate a matched study of the additional value of pressure as thresholding and placement change. An intervention that helps one recipe may add little to another, or incur a different quality cost at a new site.

We pretrain Pythia-family models from scratch to compare interventions in a controlled architecture family (Biderman et al., 2023). The detailed 14M study contains 30 conditions across eight recipes and 29 paired comparisons. *Pressure* encourages small activation magnitudes; *thresholding* applies specified zeroing maps; and *placement* selects where each acts. We compare pressure on/off at fixed nonlinearities and broader thresholding without pressure; pressured four/seven-site comparisons change complete objectives as well as placement. Model-wide activation sparsity, $\mathcal{S}_{\text{model}}$, weights actual zeros by their counted scalar multiplications. Architectural reach, R_{arch} , describes the work reached if the selected sites were entirely zero; it is not a target guaranteed attainable at useful quality.

The comparisons show that pressure’s marginal effect depends on the threshold, and that local FFN sparsity can mis-rank model-wide opportunity. At the largest 14M threshold, adding orthogonal pressure to seven-site thresholding raises $\mathcal{S}_{\text{model}}$ from 15.39% to 27.48%, against 29.95% architectural reach, for +0.1265 loss relative to the unpressured recipe and +0.6208 relative to unmodified A0. Activation distributions and operation counts explain why this recipe has greater model-wide sparsity than its four-site counterpart despite fewer pooled FFN zeros. We promote selected recipes to 70M and 410M, where the high-threshold seven-site advantage over four sites persists, while lower-threshold orderings vary. Trained recipes improve on uniform A0 clipping at aggressive targets across all three sizes; small quality budgets can instead favor clipped endpoints.

A human-guided coding-agent case study then tests execution on one RTX5090 with BF16, batch-one, uncached 2,048-token inference and full vocabulary logits. The selected implementation is $1.234\times$ faster than native PyTorch/SDPA geometrically across all 30 14M checkpoints. Sparse paths add $1.043\times$ over matched fusion with skipping disabled, helping 14 checkpoints; disabling attention skipping improves every checkpoint. Thus the native-relative gain combines fusion with a conditional benefit from sparse execution. The study connects the effect of an intervention on quality and counted products to the instruction granularity and overhead that determine whether exploiting its zeros is worthwhile.

2 RELATED WORK

2.1 SPARSIFICATION INTERVENTIONS

Post-hoc thresholding removes activations that a trained model can tolerate losing. CATS thresholds small-magnitude activations within FFNs, with both training-free and fine-tuning settings (Lee et al., 2024). TEAL extends magnitude thresholding to attention and FFN projections and uses block-wise greedy allocation to account for their different sensitivities (Liu et al., 2024). These studies show that site placement and sparsity allocation affect quality even without retraining. Our uniform clipping control provides a reference for asking how a trained recipe compares with fixed allocation at evaluation time; it does not test superiority over TEAL’s optimized allocation.

Training-time interventions alter the nonlinearity and the incentive to produce sparse activations. ReLU Strikes Back studies restoring sparse activations in language models (Mirzadeh et al., 2023); ProSparse combines ReLU, progressive ℓ_1 pressure, and threshold shifting during continued training (Song et al., 2024). ProSparse’s schedule ablations show that comparable FFN sparsity can carry different quality costs. Sparsing Law studies how sparsity changes with training data, activation function and model architecture (Luo et al., 2025), while Cetin et al. (2026) demonstrate highly sparse FFNs induced by ℓ_1 regularization during pretraining. Together, these studies show that training can reshape the quality–sparsity trade-off. We examine how pressure’s additional effect changes with the threshold and target sites, and how weighting zeros by their affected computation changes the comparison between recipes.

Extending sparsification beyond FFNs makes the choice of sites explicit. Q-Sparse applies top- k selection to attention and FFN projection inputs, reports overall sparsity and activated-parameter budgets, and ablates both the sparsifying nonlinearity and the straight-through estimator (Wang et al., 2024). Spark Transformer combines sparse FFNs with sparse attention through token-position selection and examines the role of its low-cost predictor (You et al., 2025). Projection-input sparsity and attention-position sparsity affect different work. Our seven-site intervention also thresholds feature coordinates in the query, key and value tensors used inside attention. We compare these broader nonlinearities with projection-input nonlinearities, both with and without pressure at 14M.

2.2 INFERENCE SPEEDUP

Converting activation sparsity into speedup requires an execution strategy suited to the workload. TEAL accelerates batch-one autoregressive decoding, where skipping weight accesses is valuable (Liu et al., 2024). Cetin et al. (2026) address batched matrix multiplication: their TwELL format packs sparse activations as a projection produces them and supports fused FFN computation, reducing the overhead of materializing and consuming the sparse representation. They evaluate throughput for full-sequence forward passes, a different workload from token-by-token decoding. Spark likewise couples its sparsity mechanism with an implementation that reports decoding gains (You

et al., 2025). In each setting, skipped computation and memory traffic must repay indexing, packing and launch overheads; operations that remain dense limit the full-model gain.

The execution strategy also depends on shape and sparsity structure. With larger batches, different token masks can reduce the weight-loading savings available from sparsity, as Spark’s batching analysis illustrates (You et al., 2025). Block Q-Sparse introduces structure into the mask to accommodate batched execution (Wang et al., 2024); TwELL instead makes packing and fusion compatible with tiled computation (Cetin et al., 2026). Iterative kernel generation with execution and profiling feedback offers a tool for exploring these adaptations (Ouyang et al., 2025; Luo et al., 2026). Our human-guided case study uses matched fusion and skipping controls to test when such overhead is repaid in full-model execution.

3 METHODOLOGY

3.1 SPARSIFICATION INTERVENTIONS

We study fixed elementwise thresholds that leave the number of surviving activations free to vary with the learned representation. Unlike top- k selection or its statistical approximation (Wang et al., 2024; You et al., 2025), a fixed cutoff does not target a prescribed activation count. It requires neither ranking nor input-dependent moment estimates. This motivates the design choice; an end-to-end speed advantage over selection-based methods would require a separate implementation comparison.

A sparsification intervention specifies thresholding sites $\mathcal{N}$, their nonlinearities and thresholds, and pressure sites $\mathcal{P}$ at which training encourages smaller activation magnitudes. Choosing where to apply a threshold and where to apply pressure are separate decisions, although many evaluated recipes align the two sets. Figure 1 gives the assignments; the cohort does not cross every possible combination.

We use ReLU and two thresholding nonlinearities: $G_{+,\kappa}(x) = x\mathbf{1}\{x \geq \kappa\}$ and $G_{\pm,\kappa}(x) = x\mathbf{1}\{|x| \geq \kappa\}$, with $\kappa \geq 0$. One-sided thresholding removes all negative values and positive values below κ ; symmetric thresholding removes small magnitudes while preserving the signs and values of survivors. At $\kappa = 0$, their forward maps are ReLU and the identity, respectively. Appendix A specifies boundary and derivative conventions.

The threshold is fixed within each training condition. We hypothesize that training can reshape the activation distribution around this nonlinearity, with pressure providing an additional influence. We examine quality and sparsity at the final checkpoints and compare their signed activation distributions. These endpoint measurements describe the resulting representations; they do not identify the adaptation process during training.

Pressure uses $\mathcal{L}_1$: we take the mean absolute activation in each targeted site-layer tensor, after the nonlinearity where one is applied, then average these means equally. Adding target sites therefore changes the objective and reduces the weight of each existing site-layer term. Equal λ across target sets does not mean equal per-site pressure.

Naive L1 optimizes $\mathcal{L}_{\text{task}} + \lambda\mathcal{L}_1$, clipping the combined gradient before AdamW. *Orthogonal L1* (OL1) adapts gradient-conflict projection (Yu et al., 2020; Hsieh et al., 2024) to this pressure objective. It makes a task-only AdamW update, then adds a pressure correction preconditioned by the task’s second-moment estimates. When this correction opposes the adaptive task direction, OL1 projects out its conflicting component. A positive trust budget caps its norm relative to that task direction. This limits the pressure correction without guaranteeing preserved task loss; Appendix A.2 defines the procedure.

3.2 SPARSITY AND ARCHITECTURAL REACH

Activation sparsity is the fraction of exactly zero entries at a named site. We weight zeros by their affected work using *model-wide activation sparsity*, $\mathcal{S}_{\text{model}} = N_0/N_{\text{model}}$, reported as $100\mathcal{S}_{\text{model}}\%$. One scalar multiplication in a declared transformer-block matrix product receives one zero credit if either activation operand is zero, including when both are zero. The denominator also includes the dense language-model (LM) head, which receives no zero credit. We pool integer counts before dividing and count only causally valid pairs in full, uncached sequences. Appendix B specifies the

operations and actual operands. This fraction measures a declared multiplication opportunity, not total FLOPs, memory traffic or elapsed time.

The *architectural reach*, $R_{\text{arch}} = N_{\text{reach}}(\mathcal{N})/N_{\text{model}}$, is the fraction of the declared workload structurally reached when the selected sites are entirely zero, counting overlapping effects once. It depends on the architecture, site set and sequence length, but not on a checkpoint, pressure objective or quality constraint. Observed $\mathcal{S}_{\text{model}}$ also includes natural zeros outside the selected-site reach, so R_{arch} need not bound every observation. Its gap to measured sparsity does not quantify further sparsity available at acceptable quality. Appendix B.2 defines the reach accounting.

4 EXPERIMENTAL STUDY

We pretrain randomly initialized Pythia-14M, 70M, and 410M models (Biderman et al., 2023) on MiniPile (Kaddour, 2023). The shared architecture family permits detailed comparisons at 14M and selected-recipe comparisons at the larger sizes. Quality is next-token validation cross-entropy, where lower is better. We measure loss and $\mathcal{S}_{\text{model}}$ in the same forward pass over all 338 complete 2,048-token blocks from the 500 MiniPile validation documents, excluding the 1,444-token tail. All training-result figures and tables use this evaluation.

Figure 1 locates the activation sites in the shared Pythia block and maps the eight recipes onto them. Each recipe is pretrained separately. Appendix C.1 gives exact site definitions and tensor shapes.

Within each size, matched contrasts share initialization, data order, and training budget. Pressure on/off contrasts hold the nonlinearities and threshold fixed. Unpressured A7 versus A4 at the same κ adds symmetric query/key/value thresholding. A7-OL1 versus A4-OL1 instead changes both non-linearity and pressure sites, including the weights of existing pressure terms; it compares complete recipes. A4 versus A1-H changes the additional sites and, for $\kappa > 0$, the threshold at h . At $\kappa = 0$, h has the same forward map but a different derivative at zero (Appendix A). These distinctions determine which intervention each later paired difference can identify.

The cohort contains 30 conditions at 14M and 12 at each larger size. Each trains for 712 updates and 1.493 billion input tokens; we evaluate the final checkpoint. Each condition has one training seed. Reusing initialization supports pairing, not independent replication. Local pressure varies $\lambda \in \{0.05, 0.1, 0.5, 1\}$; A4/A7 vary $\kappa \in \{0, 0.01, 0.05, 0.1, 0.5\}$. Multisite OL1 uses $\lambda = 1$, and all OL1 conditions use trust budget 1. Sweeping these settings tests whether an added intervention has a consistent effect or depends on the strength of the existing intervention. Appendix C.4 gives the optimizer, precision and learning-rate settings.

For the evaluation-only reference, we apply uniform post-hoc magnitude clipping with targets $p = 0, 0.1, \dots, 0.9$ at a, m, h, z without updating weights. Site-layer thresholds are calibrated separately on ten training blocks. This is a uniform TEAL-style control (Liu et al., 2024); it does not optimize allocation across sites. Target p is a calibration quantile, not achieved model-wide sparsity. Clipping acts after the trained transformation at h , preserving A0’s GELU, whereas A7 replaces GELU and additionally thresholds q, k, v . The comparison therefore changes reach and the final transformation as well as when sparsification is applied. The main figures show A0 clipping; Appendix D.4 includes all 540 evaluations across the 54 trained checkpoints.

4.1 QUALITY–SPARSITY TRADE-OFFS

Local pressure and broader nonlinearities offer different quality–sparsity trade-offs (Figure 2). The lowest-loss local endpoint uses naive L1 at h : at $\lambda = 1$, A1-H-L1 reaches 3.949% sparsity at loss 5.1023, compared with A0’s 5.2086. OL1 at $\lambda = 0.5$ also has lower observed loss, 5.1102, with 3.768% sparsity. A0 post-hoc clipping reaches a similar sparsity level, 3.832% at target $p = 0.3$, but with loss 5.3619. Thus, similar aggregate sparsity can carry different quality costs depending on the trained recipe and evaluation-time intervention.

Broader interventions reach much greater sparsity. A7-OL1 at $\kappa = 0.5$ attains 27.483% model-wide sparsity at loss 5.8294, an increase of 0.6208 over A0. This comparison also changes computational reach: the A0 clipping reference acts at a, m, h, z , whereas A7 additionally thresholds the internal attention operands. At 14M, these site sets have architectural reach of 12.833% and 29.952%, respectively. This comparison changes both the trained representation and the computation the se-

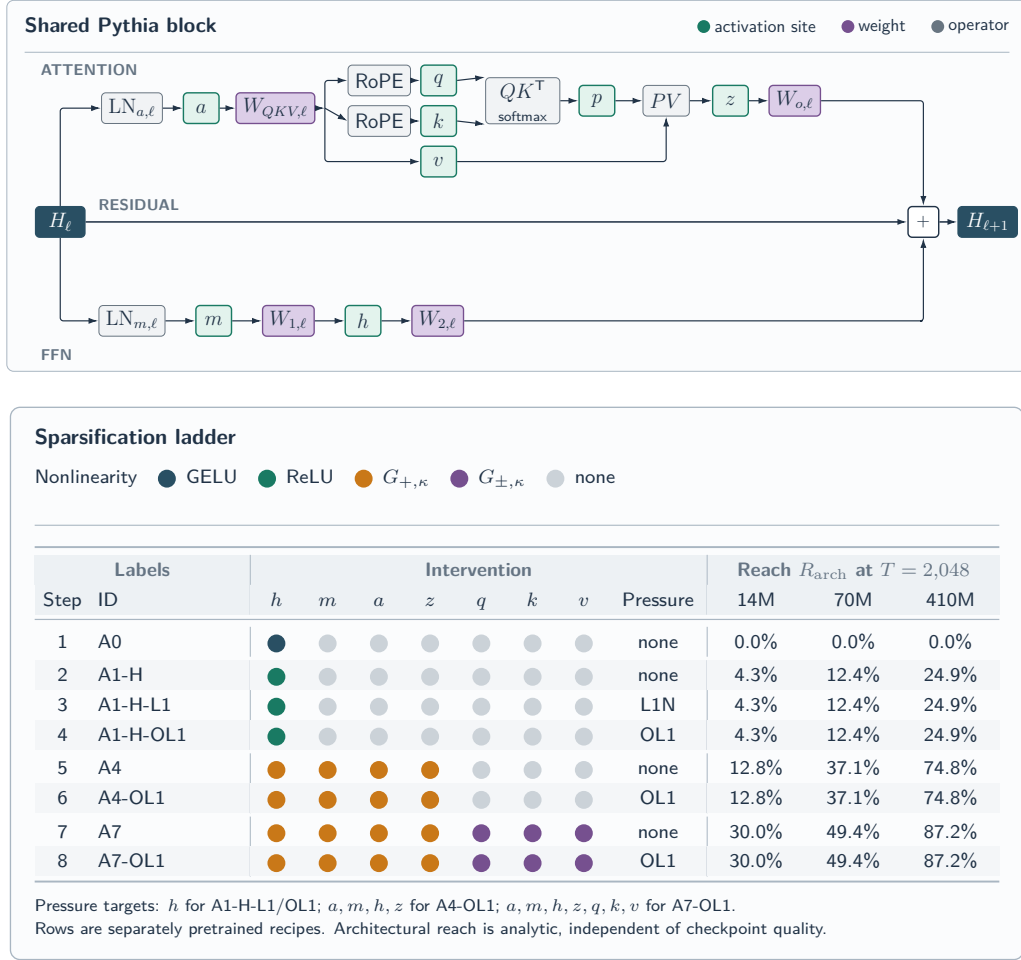


Figure 1: **Pythia architecture and sparsification recipes.** Top: the shared block and activation sites; q, k follow RoPE and z precedes the attention-output projection. Bottom: nonlinearity placement, pressure and architectural reach for the eight recipes. Selected nonlinearities replace GELU at h and act on existing activations elsewhere. L1N denotes naive L1; pressure targets every site with a selected nonlinearity in its row. The reach columns report $100R_{\text{arch}}\%$ for $T = 2,048$, rather than measured sparsity or speedup. All recipes are evaluated at 14M; 70M and 410M use A0, A1-H, A4-OL1 and A7-OL1. Rows identify separately trained recipes, not curriculum stages.

Table 1: Greatest observed sparsity with loss no greater than same-size A0 plus the stated budget. The trained column excludes A0 when identifying a sparsified recipe; “none” means no such endpoint qualifies. The last columns also consider all actual post-hoc evaluations. Budgets with identical winners are grouped. These budgets were chosen retrospectively. No kernel latency was measured for the clipped winners.

Size	Loss budget	Trained recipe	$\mathcal{S}_{\text{model}}$ (%)	Including clipping	$\mathcal{S}_{\text{model}}$ (%)
14M	0.05, 0.10, 0.20	A1-H-L1 $\lambda = 1$	3.949	A1-H-L1 $\lambda = 1, p = 0.3$	6.490
70M	0.05, 0.10	none	—	A0, $p = 0.2$	7.410
70M	0.20	A1-H	10.066	A1-H, $p = 0.3$	17.485
410M	0.05, 0.10	none	—	A0, $p = 0.2$	14.883
410M	0.20	A1-H	21.050	A1-H, $p = 0.3$	35.810

lected sites can affect. The practical question is how much of the observed opportunity remains within a specified quality budget.

Table 1 answers that question retrospectively across all 54 trained and 540 clipped evaluations. Under a +0.05 loss allowance relative to same-size A0, the greatest observed sparsities are 6.490%, 7.410% and 14.883% at 14M, 70M and 410M. All three use post-hoc clipping, including clipping of a pressure-trained checkpoint at 14M. No sparsified training recipe in the larger cohorts meets the two smallest budgets. These are selected evaluated points, not interpolated operating points; the maximum-sparsity recipe and a recipe suitable for a small loss budget need not coincide.

4.2 PAIRED INTERVENTION EFFECTS

Pressure’s additional value depends on the threshold and target set (Figure 3). Replacing GELU with ReLU alone increases model-wide sparsity by 2.714 percentage points (pp), but raises loss by 0.0611. Adding local pressure lowers observed loss relative to this ReLU control. Changing naive L1 to OL1 lowers loss at three of the four matched weights, but reverses that improvement at $\lambda = 1$; the update rule is not uniformly better across the sweep. These are single-seed comparisons of complete update rules, including their clipping and optimizer state, rather than an isolated test of conflict projection.

For A4, adding OL1 improves both loss and sparsity at $\kappa = 0$ and 0.01. At larger thresholds it adds sparsity at increasing quality cost. At $\kappa = 0.5$, the increment is 2.498 pp for +0.3783 loss. The corresponding A7 pressure addition yields 12.096 pp for +0.1265 loss, while at $\kappa = 0.1$ it yields 1.372 pp for a much smaller +0.0008 loss change. That small difference does not establish loss equivalence. Broader thresholding also contributes without pressure: A7 minus A4 at $\kappa = 0.5$ adds 5.171 pp for +0.0432 loss.

Pressure therefore needs to be evaluated at its actual threshold and sites. The on/off comparisons establish threshold-dependent effects within each defined objective. The larger gain under A7 involves more pressure targets and a different objective normalization; it cannot be assigned solely to broader nonlinearity placement or to pressure at q, k, v . We next examine the distributions learned by these complete recipes.

4.3 RESHAPING ACTIVATION DISTRIBUTIONS

More FFN zeros need not mean more model-wide sparsity (Figure 4). At $\kappa = 0.5$, A4-OL1 places 99.31% of pooled FFN elements at h, m exactly at zero, compared with 93.64% for A7-OL1. Their attention distributions differ more sharply: A4-OL1 has a narrow nonzero peak and only 0.22% exact zeros, whereas A7-OL1 has 95.60% exact zeros and nonzero density outside the symmetric threshold gap. Each panel reports these zero masses beside the nonzero density curves; Table 9 also gives the tails. A narrow nonzero density peak is not itself a large exact-zero probability.

A4 does not directly threshold or pressure q, k, v , yet their distributions change substantially under the complete A4-OL1 intervention. The comparison with A0 does not isolate pressure’s contribution from the other training changes. A7 directly intervenes at these attention sites, so its reshaping there has a different interpretation.

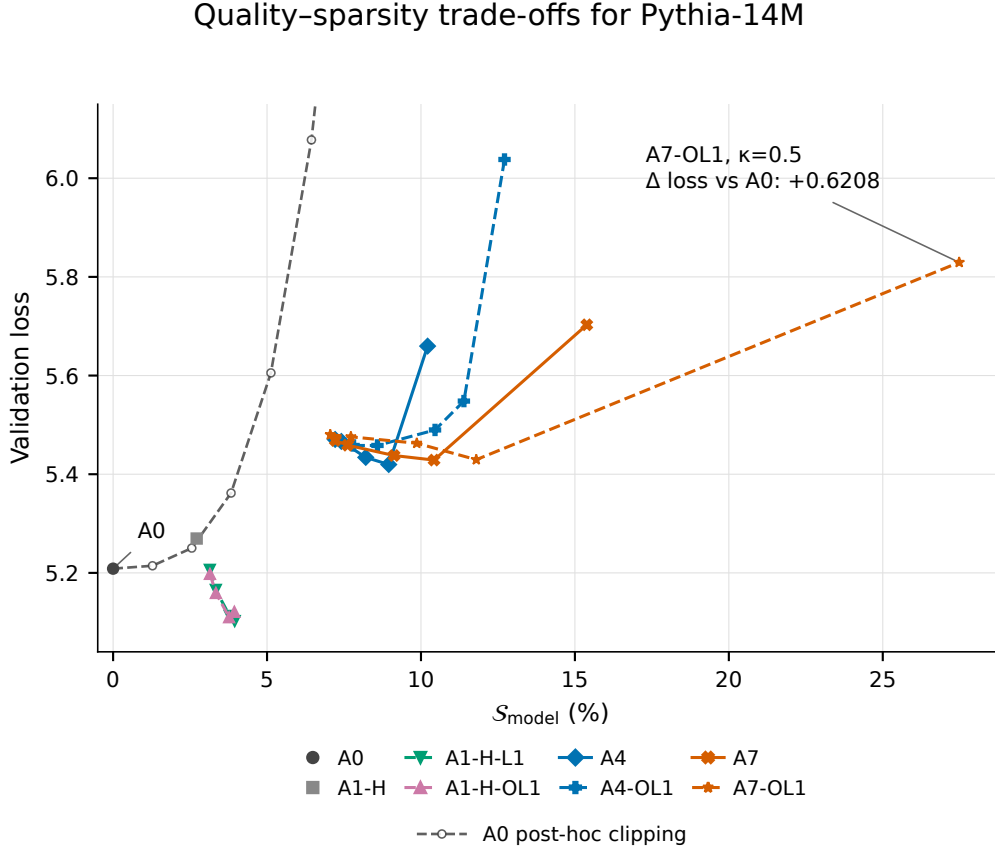


Figure 2: **Quality-sparsity trade-offs from training interventions and post-hoc clipping.** Validation loss versus model-wide sparsity S_{model} for Pythia-14M. Filled markers show all 30 separately trained checkpoints: A0, A1-H, and the A1-H-L1, A1-H-OL1, A4, A4-OL1, A7 and A7-OL1 sweeps. Solid colored curves denote unpressured variants; dashed colored curves denote pressure variants. The gray dashed path with open markers applies evaluation-only magnitude clipping at a, m, h, z to A0, with calibration targets $p = 0, 0.1, \dots, 0.9$. Six of its ten evaluations lie inside the displayed loss range; Appendix D.4 supplies the full-range view. Lines connect evaluated settings, not fitted Pareto envelopes. Local pressure reaches lower loss with modest sparsity; A7-OL1 reaches greater sparsity at a quality cost and with broader computational reach. All points use the matched protocol in Section 4.

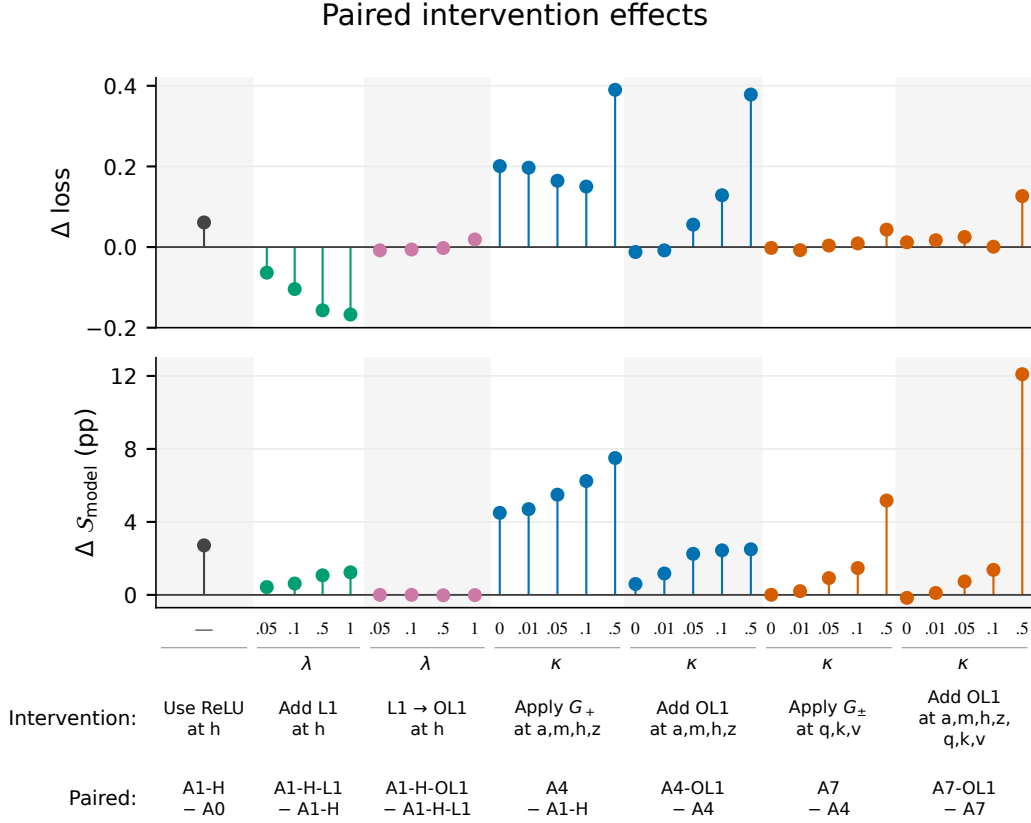


Figure 3: **Pressure’s added value depends on the threshold and target sites.** Changes in validation loss (top; negative is better) and $100S_{\text{model}}$ (bottom; percentage points) for 29 matched Pythia-14M comparisons. Each delta is treatment minus reference, as specified in the **Paired:** row; the **Intervention:** row gives the operation and sites. Labels give λ or κ in increasing order with ordinal spacing; the dash denotes the unswept ReLU replacement. L1-to-OL1 comparisons share λ ; A4/A7 pressure additions and the Q/K/V thresholding addition share κ . Other comparisons use the fixed A0 or A1-H control. Pairs share initialization, data order and budget, with one training seed and reused controls. Pressure additions hold the nonlinearities fixed; A4 versus A1-H also changes the h threshold above zero. These are trained endpoint differences, not independent replicates or successive training updates.

Table 2: Why local and model-wide sparsity rank the 14M recipes differently at $\kappa = 0.5$. Every contribution uses the same full-model denominator, including the dense LM head with no zero credit. Projection contributions sum QKV, attention output and both FFN projections; these are counted scalar products, not time or FLOPs saved. Full operation entries are in Appendix D.3.

Recipe	Projections (pp)	$QK + PV$ (pp)	Total $\mathcal{S}_{\text{model}}$ (%)
A4-OL1	12.657	0.057	12.713
A7-OL1	10.709	16.773	27.483

The operation counts show why this distinction matters model-wide. At the same threshold, QK and PV contribute 16.77 sparsity points under A7-OL1, versus 0.06 under A4-OL1 (Table 2). Projection contributions together fall from 12.66 to 10.71 points, but the additional 16.72 attention points more than offset that reduction. Local FFN sparsity therefore cannot rank these recipes by their counted model-wide opportunity.

Separate BF16 kernel diagnostics also reveal entirely zero projection-input vectors. At $\kappa = 0.5$, the all-zero row fractions at h and z are 83.72% and 97.15% for A4-OL1, versus 63.12% and 91.20% for A7-OL1. Appendix D.2 reports per-layer nonzero counts and this measurement’s precision. Zero inputs do not imply zero residual-branch outputs, because biases may remain; these aggregate records also cannot establish context sensitivity or loss of branch function.

4.4 TRANSFER ACROSS MODEL SIZES

The high-threshold A7-OL1 advantage persists at all three model sizes (Figure 5). At $\kappa = 0.5$, A7-OL1 has both lower loss and greater model-wide sparsity than A4-OL1 at 14M, 70M and 410M (Table 3). The zero-threshold ordering does not transfer unchanged: A4-OL1 is better on both axes at 14M and 70M, whereas A7-OL1 is better at 410M. Persistence concerns a specific recipe relationship, not the entire threshold response.

Raw sparsity must also be read against the changing workload. A7’s architectural reach rises from 29.95% to 49.42% and 87.25% as the dense LM head accounts for a smaller share of counted products. The bottom row uses the same A7 denominator for every curve within a size: $\mathcal{S}_{\text{block}} = \mathcal{S}_{\text{model}}/R_{\text{arch}}(\text{A7})$. This identity holds because A7 reaches all counted block operations. Block-only sparsity removes the dense-head share while retaining the different weights of operations within the blocks.

This is transfer of a complete-recipe comparison. The larger cohorts lack the no-pressure A4/A7 controls needed to isolate pressure’s contribution. They share a token budget rather than tokens per parameter, and 410M uses a lower peak learning rate. Its A0 loss is also higher than 70M’s under this budget; the plot should not be read as a quality scaling law.

Table 3: Complete-recipe contrasts at the boundary thresholds. Entries are A7-OL1 minus A4-OL1 at the same size and trained threshold. Negative loss changes and positive sparsity changes favor A7-OL1. All entries use complete validation. Table 8 reports all five thresholds.

Size	κ	Δloss	$\Delta \mathcal{S}_{\text{model}}$ (pp)
14M	0	+0.0219	-0.756
14M	0.5	-0.2086	+14.769
70M	0	+0.1358	-2.110
70M	0.5	-0.1736	+5.006
410M	0	-0.2658	+2.783
410M	0.5	-0.0703	+9.024

Exact-zero mass and nonzero activation distributions

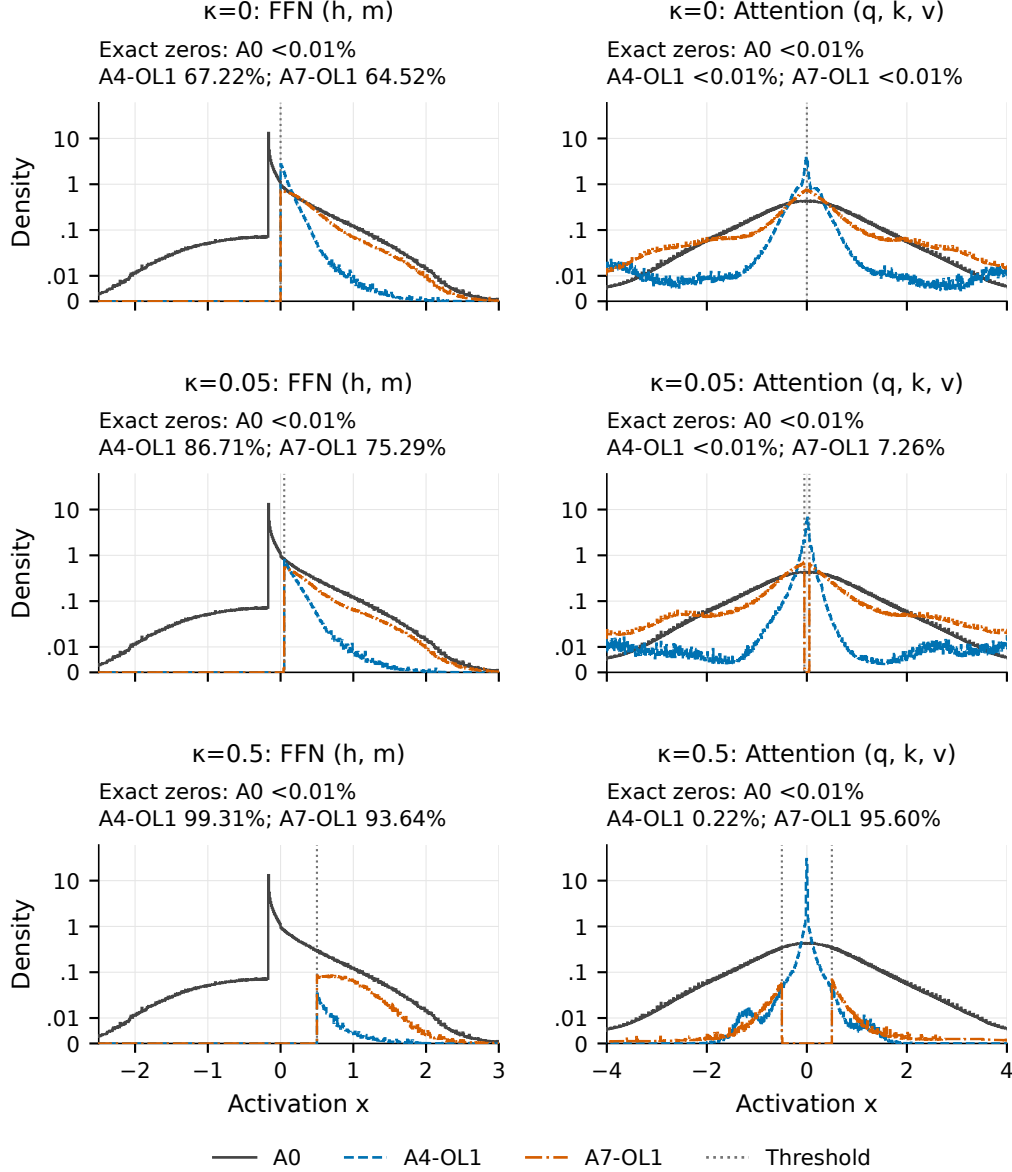


Figure 4: **Interventions reshape FFN and attention distributions differently. Exact zeros are reported above each panel and excluded from the nonzero curves.** Columns pool FFN (h, m) and attention (q, k, v) activations; rows use trained thresholds $\kappa = 0, 0.05, 0.5$. A0 is repeated across rows. Counts are pooled across all layers and validation blocks, weighting $h : m$ by 80:20 and $q : k : v$ equally. Density divides nonzero bin counts by all captured elements and bin width, without renormalizing the visible range. The vertical scale is symlog, linear below 0.01 and logarithmic above it; plotted areas do not represent probability mass. Dotted lines mark $+\kappa$ for A4/A7 FFN thresholds and $\pm\kappa$ for A7 attention thresholds. Q/K are measured after RoPE. A4 leaves q, k, v without direct thresholding or pressure; A7 targets them directly. The seven checkpoints use no additional post-hoc clipping.

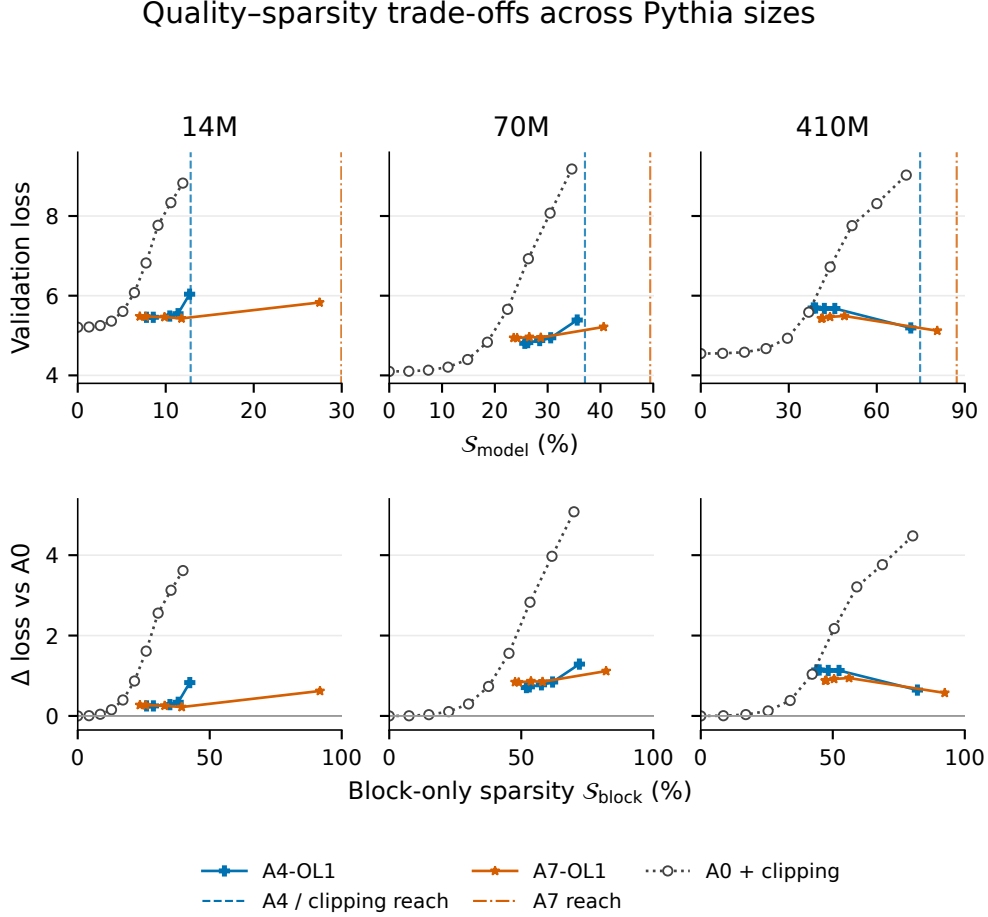


Figure 5: **The high-threshold A7-OL1 advantage persists across model sizes.** Columns compare 14M, 70M and 410M. Solid curves connect separately trained A4-OL1 and A7-OL1 checkpoints at $\kappa = 0, 0.01, 0.05, 0.1, 0.5$; dotted open-marker curves show A0 post-hoc clipping at a, m, h, z for targets $p = 0, 0.1, \dots, 0.9$. The 60 evaluations appear in both rows. Top: validation loss versus $100S_{\text{model}}$; vertical guides indicate A4/clipping-site and A7 architectural reach. Bottom: loss minus unmodified same-size A0 versus $100S_{\text{block}}$, where $S_{\text{block}} = S_{\text{model}}/R_{\text{arch}}(\text{A7})$ for **every** curve. The A7 reference reach values are 29.95%, 49.42% and 87.25%; the A4 guide is not the normalization denominator. At $\kappa = 0.5$, A7-OL1 improves both axes over A4-OL1 at all sizes; the ordering at smaller thresholds varies. Table 3 gives the boundary-threshold comparisons. These are complete-recipe comparisons with matched token budgets, unequal tokens per parameter and a lower 410M learning rate.

4.5 REALIZING SPARSITY WITH SPECIALIZED INFERENCE KERNELS

Realizing activation sparsity requires kernels matched to the model’s shapes, nonlinearities and hardware. The released TwELL kernels (Cetin et al., 2026) accelerated their supported workload, but the unchanged small-width primitive failed numerical qualification for Pythia-14M. A shape- and sign-adapted descendant, P0, qualifies on only four of our 30 checkpoints, with geometric mean speedup $0.8910\times$ on that subset (Appendix D.5). These are compatibility results for the tested shapes and implementation; the four-checkpoint mean is not a fair aggregate comparator for our full cohort.

To make this specialization practical within an architecture study, we use a human-guided coding-agent loop to propose, test and refine implementations. Execution and profiling feedback provide a basis for this approach (Ouyang et al., 2025; Luo et al., 2026). We replay 42 eligible proposals from the Codex search against native PyTorch scaled dot-product attention (SDPA) in CUDA graphs on one RTX5090. The workload is BF16, batch-one, uncached 2,048-token inference with full vocabulary logits. Qualification requires logits and loss to agree with the native reference within specified tolerances on complete validation. The recorded agent default is *gpt-6-astra*, without a verified served model for every request; this single trajectory does not measure agent productivity against human-only development.

The selected implementation K050 qualifies on all 30 checkpoints in the 14M cohort, with geometric mean native-relative speedup $1.2340\times$. The matched search retrospective reaches $1.7830\times$ on its fixed A7-OL1, $\kappa = 0.5$ checkpoint; Appendix D.5 retains the search history. Across the final cohort, canonical FP16 model-wide sparsity is associated with BF16 native-relative speedup (descriptive OLS $R^2 = 0.8167$). Topology, weights, quality and execution overhead vary across these checkpoints, so this association alone cannot attribute the gain to skipping zero products.

Ablations distinguish fusion from the contribution of sparse paths. With all sparse paths disabled, the fused implementation already achieves $1.1829\times$ geometric mean speedup over native execution (Figure 6). Relative to this control, K050 is $1.311\times$ as fast at the search checkpoint and $1.043\times$ as fast geometrically across the cohort, helping 14 of 30 checkpoints. These incremental factors derive from separately measured native-relative timings; they do not credit fusion as a sparsity gain.

Attention skipping is unprofitable in this workload: disabling it improves every checkpoint, raising the mean to $1.2506\times$. The attention kernel can skip a matrix-multiply instruction when either complete operand fragment is zero. At the search checkpoint, it bypasses 58.0% of eligible *QK* and 67.2% of *PV* instructions, yet remains slower than executing those paths densely. Partially zero fragments can still require an instruction, and detection and control have a cost. Scalar zero opportunity alone is therefore insufficient to select a profitable execution path.

The incremental sparse-path factor has a weaker association with canonical FP16 sparsity than native-relative speedup does ($R^2 = 0.5037$ versus 0.8167). Removing the searched checkpoint gives $R^2 = 0.4080$ for the incremental factor; removing its entire A7-OL1 family gives 0.4910 . Appendix D.5.1 reports all leave-family and within-family checks. These descriptive comparisons retain the FP16/BF16 mismatch and are not prospective prediction tests.

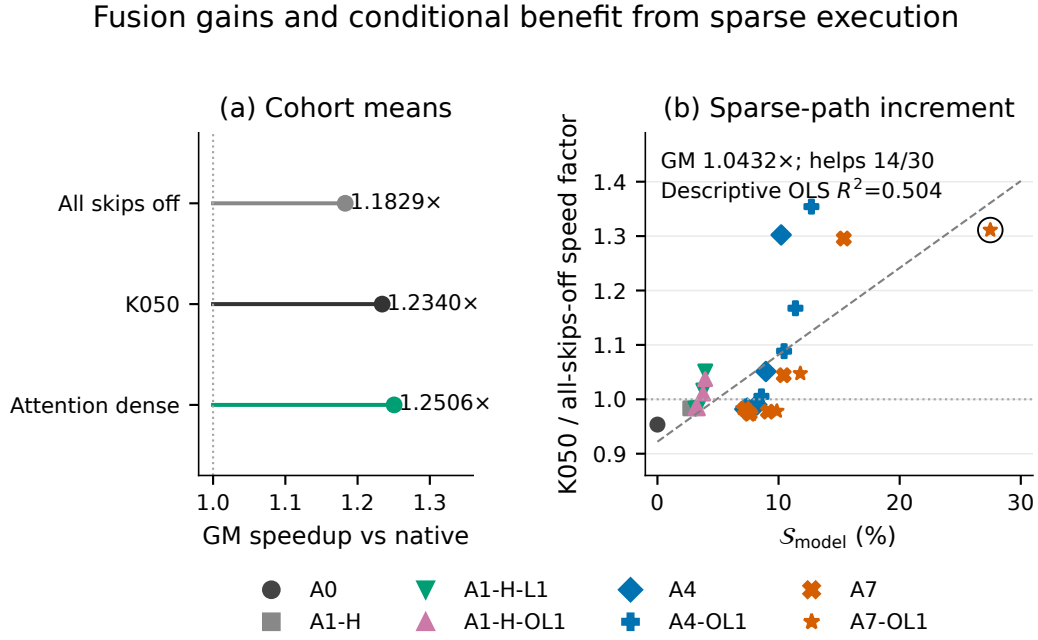


Figure 6: **Fusion gains coexist with conditional benefit from sparse paths.** (a) Equal-checkpoint geometric mean speedups on all 30 Pythia-14M checkpoints, each relative to its native SDPA CUDA-graph baseline. Attention-dense execution is faster than K050 on every checkpoint. (b) K050’s native-relative speedup divided by that of all-skips-off; values above one favor sparse execution. The circled point is the searched A7-OL1, $\kappa = 0.5$ checkpoint. Symbols identify recipes; the dashed line is descriptive OLS with an intercept. Timings use one RTX5090, BF16, batch one, uncached 2,048-token sequences and full vocabulary logits; qualification covers all 338 validation blocks. The horizontal axis uses canonical FP16 counts. Axes have restricted ranges; neither the fit nor these ratios imply a causal or general scaling law.

5 DISCUSSION AND CONCLUSION

An intervention should be evaluated against the recipe to which it is added. In the matched 14M comparisons, pressure can improve both observed loss and sparsity at one threshold, then exchange quality for additional zeros at another. Broader intervention at internal attention can also increase model-wide zero-product counts while reducing pooled FFN sparsity. Assessing an addition therefore requires its paired quality change and the operations affected, alongside the number of zero activations.

Recipe comparisons, model selection under a loss budget, and kernel selection require different references. The high-threshold seven-site pressured recipe improves on its four-site counterpart at all three sizes, yet neither recipe supplies the best point under the smallest tested A0-relative loss budgets. Similarly, a kernel can accelerate a sparse checkpoint relative to native execution while some of its skipping paths make it slower. A favorable paired recipe result or a high sparsity percentage alone does not settle the other selection decisions.

Architectural reach identifies which products the selected sites can affect; actual-operand counts measure zeros in the learned model; kernel instrumentation measures skipped instructions; and timing determines whether skipping pays. The attention ablation separates these quantities concretely: many eligible instructions are skipped, but dense attention is faster on every checkpoint. Reporting native, matched-fusion and sparse-path controls is necessary to identify where the observed acceleration comes from.

These conclusions concern one-seed MiniPile pretraining at a fixed token budget, with a lower learning rate at 410M and selected complete recipes at the larger sizes. Uniform four-site clipping is a limited post-hoc comparator. Distribution diagnostics cover 14M; the kernel study uses one RTX5090 and full-sequence inference, with canonical FP16 counts and BF16 timing. Two high-value questions remain: whether the placement relationships replicate when existing pressure coefficients are held fixed, and whether the same execution trade-off persists at larger widths.

AI USE STATEMENT

AI assistance supported code for the reported reductions, inspection of experiment records, interpretation of results, literature checks, preparation of figures and tables, and drafting and editing this manuscript. The kernel case study additionally used a human-guided coding-agent loop for implementation, testing and optimization. Its recorded default model was *gpt-6-astra*; per-request served-model identities were not verified. Reported measurements are tied to retained execution records. Numerical reductions were checked against source counts and timing samples, and kernel claims use the qualification procedure in Appendix D.5.

REFERENCES

- Stella Biderman, Hailey Schoelkopf, Quentin Gregory Anthony, Herbie Bradley, Kyle O’Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar van der Wal. Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pp. 2397–2430. PMLR, 2023. URL <https://proceedings.mlr.press/v202/biderman23a.html>.
- Edoardo Cetin, Stefano Peluchetti, Emilio Castillo, Akira Naruse, Mana Murakami, and Llion Jones. Sparser, Faster, Lighter Transformer Language Models. *arXiv preprint arXiv:2603.23198*, 2026. URL <https://arxiv.org/abs/2603.23198>.
- Yu-Guan Hsieh, James Thornton, Eugene Ndiaye, Michal Klein, Marco Cuturi, and Pierre Ablin. Careful with that Scalpel: Improving Gradient Surgery with an EMA. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 19085–19100. PMLR, 2024. URL <https://proceedings.mlr.press/v235/hsieh24a.html>.

- Jean Kaddour. The MiniPile Challenge for Data-Efficient Language Models. *arXiv preprint arXiv:2304.08442*, 2023. URL <https://arxiv.org/abs/2304.08442>.
- Donghyun Lee, Je-Yong Lee, Genghan Zhang, Mo Tiwari, and Azalia Mirhoseini. CATS: Contextually-Aware Thresholding for Sparsity in Large Language Models. In *Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=v3w2a7EInO>.
- James Liu, Pragaash Ponnusamy, Tianle Cai, Han Guo, Yoon Kim, and Ben Athiwaratkun. Training-Free Activation Sparsity in Large Language Models. *arXiv preprint arXiv:2408.14690*, 2024. URL <https://arxiv.org/abs/2408.14690>.
- Ilya Loshchilov and Frank Hutter. Decoupled Weight Decay Regularization. In *International Conference on Learning Representations*, 2019. URL <https://arxiv.org/abs/1711.05101>.
- Mao Luo, Hongbin Li, Feng Lin, Hanling Yi, and Zhe Huang. Agentic Kernel Optimization: Generating State-of-the-Art GPU Kernels Without Hand-Written CUDA. *arXiv preprint arXiv:2608.14560*, 2026. URL <https://arxiv.org/abs/2608.14560>.
- Yuqi Luo, Chenyang Song, Xu Han, Yingfa Chen, Chaojun Xiao, Xiaojun Meng, Liqun Deng, Jian-sheng Wei, Zhiyuan Liu, and Maosong Sun. Sparsing Law: Towards Large Language Models with Greater Activation Sparsity. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 41311–41330. PMLR, 2025. URL <https://proceedings.mlr.press/v267/luo25i.html>.
- Iman Mirzadeh, Keivan Alizadeh, Sachin Mehta, Carlo C. Del Mundo, Oncel Tuzel, Golnoosh Samei, Mohammad Rastegari, and Mehrdad Farajtabar. ReLU Strikes Back: Exploiting Activation Sparsity in Large Language Models. *arXiv preprint arXiv:2310.04564*, 2023. URL <https://arxiv.org/abs/2310.04564>.
- Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. KernelBench: Can LLMs Write Efficient GPU Kernels? In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 47356–47415. PMLR, 2025. URL <https://proceedings.mlr.press/v267/ouyang25a.html>.
- Chenyang Song, Xu Han, Zhengyan Zhang, Shengding Hu, Xiyu Shi, Kuai Li, Chen Chen, Zhiyuan Liu, Guangli Li, Tao Yang, and Maosong Sun. ProSparse: Introducing and Enhancing Intrinsic Activation Sparsity within Large Language Models. *arXiv preprint arXiv:2402.13516*, 2024. URL <https://arxiv.org/abs/2402.13516>.
- Hongyu Wang, Shuming Ma, Ruiping Wang, and Furu Wei. Q-Sparse: All Large Language Models can be Fully Sparsely-Activated. *arXiv preprint arXiv:2407.10969*, 2024. URL <https://arxiv.org/abs/2407.10969>.
- Chong You, Kan Wu, Zhipeng Jia, Lin Chen, Srinadh Bhojanapalli, Jiaxian Guo, Utku Evci, Jan Wassenberg, Praneeth Netrapalli, Jeremiah J. Willcock, Suvinay Subramanian, Felix Chern, Alek Andreev, Shreya Pathak, Felix Yu, Prateek Jain, David E. Culler, Henry M. Levy, and Sanjiv Kumar. Spark Transformer: Reactivating Sparsity in FFN and Attention. *arXiv preprint arXiv:2506.06644*, 2025. URL <https://arxiv.org/abs/2506.06644>.
- Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient Surgery for Multi-Task Learning. In *Advances in Neural Information Processing Systems*, 2020. URL <https://arxiv.org/abs/2001.06782>.

A INTERVENTION DEFINITIONS

A.1 THRESHOLDING NONLINEARITIES

For finite $\kappa \geq 0$, both thresholding nonlinearities retain equality and leave retained values unchanged. The comparison mask is detached: the derivative with respect to the input is one for retained entries and zero for rejected entries. Consequently, $G_{+,0}$ and ReLU have identical forward values but differ at zero, where their derivatives are one and zero, respectively. $G_{\pm,0}$ is exactly the identity in both values and gradients. These nonlinearities use neither a straight-through estimator nor a top- k selection rule. Unselected sites retain their original transformations.

A.2 ACTIVATION PRESSURE AND OL1

For the J targeted site-layer tensors A_j , containing n_j entries each, the pressure objective is $\mathcal{L}_1 = J^{-1} \sum_{j=1}^J n_j^{-1} \sum_{i=1}^{n_j} |A_{j,i}|$. The tensors contain the executed site activations, after the non-linearity where one is applied. Absolute values and means are evaluated in float32. Each tensor receives equal weight regardless of its size; adding pressure sites changes both the objective and the weight of existing targets. Task and pressure losses are averaged over the same accumulation microbatches. For naive L1, the accumulated gradient of $\mathcal{L}_{\text{task}} + \lambda \mathcal{L}_1$ is globally norm-clipped before AdamW (Loshchilov & Hutter, 2019).

For OL1, let g_t and g_1 be the accumulated task and unweighted pressure gradients. We first clip g_t by its global norm and perform AdamW using only that clipped task gradient. Let $\hat{m}_t$ and $\hat{v}_t$ be its newly updated, bias-corrected moments. On parameters having both gradients and initialized optimizer state, define the adaptive task and pressure vectors $u = \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon_A)$ and $w = g_1 / (\sqrt{\hat{v}_t} + \epsilon_A)$. The pressure gradient is unclipped and has no first-moment buffer. Inner products and norms below pool all these eligible parameters. With $c = \langle u, w \rangle$, $q = \|u\|_2^2$, and numerical stabilizer $\epsilon > 0$, the conflict-conditioned projection is

$$\tilde{w} = \begin{cases} w - \frac{c}{q + \epsilon} u, & c < 0 \text{ and } q > \epsilon, \\ w, & \text{otherwise.} \end{cases} \quad (1)$$

For pressure weight $\lambda \geq 0$ and required budget $b > 0$, define the norm ratio $r = \lambda \|\tilde{w}\|_2 / (\|u\|_2 + \epsilon)$. Set $s = \min\{1, b/(r + \epsilon)\}$ when $r > 0$, or $s = 1$ otherwise. After AdamW, each eligible parameter group p , with learning rate α_p , receives $\theta_p \leftarrow \theta_p - \alpha_p \lambda s \tilde{w}_p$. The experiments' clipping thresholds, b , and λ are condition parameters.

This procedure is related to gradient-conflict projection (Yu et al., 2020) and the use of smoothed primary directions (Hsieh et al., 2024), but defines a particular correction in AdamW-preconditioned coordinates. The stabilized projection is only approximately orthogonal under conflict. Its geometry and budget precede multiplication by group learning rates and exclude decoupled weight decay. They therefore do not guarantee task-loss descent or preservation.

B SPARSITY ACCOUNTING

B.1 EXACT COUNTERS

Activation sparsity at site s is $\mathcal{Z}_s = \sum_{\ell, e, i} \mathbf{1}\{A_{s,i}^{(\ell, e)} = 0\} / \sum_{\ell, e} |A_s^{(\ell, e)}|$, where ℓ indexes layers, e evaluation batches, i tensor entries, and $|A|$ the number of elements in tensor A . Near-zero mass replaces equality by $|A_{s,i}| \leq \epsilon$ for a stated tolerance; it does not contribute to exact-zero counts unless the value is zero. All aggregate fractions pool integer numerators and denominators across batches and layers before division.

The atomic unit of model-wide accounting is one scalar multiplication in a matrix product. If $X \in \mathbb{R}^{n \times p}$ feeds a linear projection with q outputs, the count is $N = npq$, with $N_0 = q \sum_{i,j} \mathbf{1}\{X_{ij} = 0\}$. Zero weights receive no credit. For the multihead attention graph specified in Appendix C.1, both operands are activations, so a product is credited once if either is zero. Within one block and

evaluation batch, for query position t , key position $u \leq t$, and within-head coordinate j ,

$$N_{0,QK} = \sum_{b,h,t} \sum_{u \leq t} \sum_j \mathbf{1}\{Q_{bhtj} = 0 \vee K_{bhuj} = 0\}, \quad (2)$$

$$N_{0,PV} = \sum_{b,h,t} \sum_{u \leq t} \sum_j \mathbf{1}\{P_{bhtu} = 0 \vee V_{bhuj} = 0\}. \quad (3)$$

Here P is the causal softmax probability tensor; the model numerator sums these counts over blocks and evaluation batches. Future masked positions are excluded from both counts, while zeros on valid positions, including floating-point underflow, are included. Counts use the operands actually consumed by each matrix product; upstream masks do not determine those operands' zeros after intervening transformations. Earlier value positions participate in more causal pairs, so local value sparsity alone does not determine PV sparsity.

B.2 ARCHITECTURAL REACH AND NORMALIZATION

For a specified graph and workload, let N_o count the valid scalar multiplications in block operation family o , pooled over layers and examples. Then $N_{\text{model}} = \sum_o N_o + N_{\text{head}}$, while N_0 sums the corresponding zero-operand counts only in block operations. The block-only view is $\mathcal{S}_{\text{block}} = N_0 / \sum_o N_o$. Both sparsities are fractions; percentages multiply them by 100. Embedding lookup, normalization, nonlinearities, softmax, RoPE, bias and residual additions, and data movement are outside this accounting.

To construct the reach numerator, count products guaranteed to have a zero activation operand as a structural consequence of 100% activation sparsity at the selected sites, independently of checkpoint values or natural zeros elsewhere. Include exact zero-preserving propagation under the declared graph rules. Products reached through more than one site receive one credit. The resulting count is $N_{\text{reach}}(\mathcal{N})$. Division by the same N_{model} gives R_{arch} . Natural zeros outside this selected-site reach can contribute to observed $\mathcal{S}_{\text{model}}$, so architectural reach is not an upper bound on every observation. Its gap to an observation does not measure sparsity available without quality loss. Appendix C.2 specializes the denominator and reach rules to the Pythia graph used in the experiments.

C EXPERIMENTAL SETUP DETAILS

C.1 PYTHIA SITES AND NONLINEARITIES

Pythia uses parallel attention and FFN branches with separate LayerNorms. For batch size B , sequence length T , residual width d , FFN width d_f , and H heads, let $d_h = d/H$. Table 4 identifies the actual operands recorded after the selected nonlinearity. The same site mapping is applied in every layer, with one common threshold per condition. Figure 1 in Section 4 shows the shared block and its recipe and pressure assignments.

Table 4: Intervention sites and the matrix products that consume them directly.

Site	Shape	Consuming operation	Placement
a	$B \times T \times d$	QKV projection	After attention LayerNorm
m	$B \times T \times d$	FFN up projection	After FFN LayerNorm
h	$B \times T \times d_f$	FFN down projection	Nonlinearity output
q, k	$B \times H \times T \times d_h$	Attention scores	After partial RoPE
v	$B \times H \times T \times d_h$	Probability-value	After the QKV split
z	$B \times T \times d$	Attention output	Concatenated context, before output projection

At h , the selected nonlinearity replaces GELU. Other sites retain their original activations unless a thresholding nonlinearity is applied. At zero threshold, A7's additional symmetric query/key/value nonlinearities are exact identities in both values and gradients. They therefore do not change A4's mathematical forward map in the absence of pressure. A7-OL1 still differs from A4-OL1 in its pressure targets and their equal-tensor normalization.

Query and key counts use actual post-RoPE operands: rotation can repopulate an individual zero coordinate. Likewise, the output projection uses actual post-nonlinearity context z , rather than inferring its zeros from the value mask.

C.2 ARCHITECTURE-SPECIFIC COUNTS AND ARCHITECTURAL REACH

The operation inventory comprises QKV projection, attention scores (QK), probability-value products (PV), attention-output projection, and the two FFN projections. For one uncached, unpadded sequence, each block contributes $3Td^2$ products from QKV projection, Td^2 from the attention-output projection, Tdd_f from each FFN projection, and $dT(T+1)/2$ from each of QK and PV . Thus, for L blocks and vocabulary size V_{vocab} ,

$$\begin{aligned} C_{\text{block}} &= T(4d^2 + 2dd_f) + dT(T+1), \\ C_{\text{model}} &= LC_{\text{block}} + TdV_{\text{vocab}}. \end{aligned} \quad (4)$$

Over B sequences, $N_{\text{model}} = BC_{\text{model}}$; the dense head contributes only to the denominator, as defined in Section 3.2.

Let $\mathcal{O}(\mathcal{N})$ be the union of operations reachable when the selected sites have 100% activation sparsity: a reaches QKV; m reaches FFN up; h reaches FFN down; either q or k reaches QK ; v reaches PV and attention output; and z reaches attention output. Each operation is counted once. The v rule includes the exact closure $V = 0 \Rightarrow PV = 0$. No further propagation is assumed: QKV biases can survive $a = 0$, and zero scores do not imply zero softmax probabilities. For the same site set in every layer, $N_{\text{reach}} = BL \sum_{o \in \mathcal{O}(\mathcal{N})} C_o$, where C_o is that operation’s per-sequence count. Division by N_{model} gives R_{arch} .

With Pythia’s $d_f = 4d$, the per-block reachable counts for A0, A1-H, A4, and A7 are 0, $4Td^2$, $12Td^2$, and C_{block} , respectively. The ladder evaluates these formulas at $T = 2,048$, $V_{\text{vocab}} = 50,304$, and the pinned (L, d) configurations (6, 128), (6, 512), and (24, 1,024).¹ No checkpoint enters this calculation. For example, A0 has zero selected-site reach but may have positive observed sparsity from natural zeros.

C.3 EVALUATION SCOPE

Quality and logical counts are paired in the same eager-attention evaluation of a checkpoint, with its declared nonlinearities and evaluation precision. The complete MiniPile validation split contains 500 documents. Concatenation and packing produce 338 complete blocks of 2,048 tokens (692,224 input tokens); the final 1,444-token tail is excluded. Each block contributes 2,047 next-token targets to cross-entropy, while product accounting includes all 2,048 input positions. Loss is averaged over the equally sized blocks, and sparsity is count-pooled. This workload uses no key-value cache. Condition-specific training budgets, optimizer settings, seeds, and checkpoint selection accompany their results; they are not implied by the ladder’s list of candidate conditions.

C.4 REALIZED TRAINING PROTOCOL

We use MiniPile and the Pythia-14M-deduped tokenizer for every size. Each document is encoded without added special tokens, followed by one end-of-sequence token. Concatenated tokens are packed into nonoverlapping 2,048-token blocks. Training uses a seeded permutation of the block indices; validation retains source order. The training split contains 1,000,000 documents and 1,491,711,416 tokens, yielding 728,374 complete blocks and an excluded 1,464-token tail. The supplementary `protocol.json` records immutable dataset and tokenizer revisions, cache hashes and implementation identities.

All conditions start from random weights drawn from the pinned Pythia architecture configurations; released pretrained weights are not used. Model initialization and data-order seeds are both 1234, with one training seed per condition. Parameter sweeps measure intervention variability rather than variation over independent training seeds. All quality comparisons reuse the same validation split, without a separate confirmation set. Within a size, the conditions share the initial parameter hash and

¹Architecture configurations: EleutherAI’s Pythia-14M-deduped, Pythia-70M-deduped, and Pythia-410M-deduped. Links identify immutable configuration revisions; released weights are not used.

training schedule. The 30-condition 14M cohort contains A0, A1-H, four weights each for A1-H-L1 and A1-H-OL1, and five thresholds each for A4, A4-OL1, A7 and A7-OL1. Each larger cohort contains A0, A1-H, and the five-threshold A4-OL1/A7-OL1 sweeps. Local pressure weights are $\{0.05, 0.1, 0.5, 1\}$; thresholds are $\{0, 0.01, 0.05, 0.1, 0.5\}$. Multisite OL1 uses $\lambda = 1$, with pressure at all four or seven selected sites. Every OL1 condition uses trust budget 1 and projection stabilizer $\epsilon = 10^{-12}$.

Training uses AdamW with $\beta = (0.9, 0.95)$, $\epsilon = 10^{-8}$, weight decay 0.1 and global task-gradient clipping at norm 1. Naive L1 clips the combined objective gradient instead. The learning rate warms up linearly for eight updates and follows cosine decay to one tenth of its peak. Parameters and optimizer state are FP32; forward computation uses FP16 autocast with dynamic loss scaling. Attention and hidden dropout are zero. Table 5 gives size-dependent settings. The global batch is 1,024 sequences of 2,048 tokens, for 712 optimizer updates and 1,493,172,224 input tokens. This wraps 714 blocks beyond the 728,374 complete blocks in one packed training pass. We use the final checkpoint without validation-based checkpoint selection. The lower 410M learning rate is part of the realized protocol, so size alone is not the only cross-scale change.

Table 5: Size-dependent training settings. Tokens per parameter use the common realized input-token budget; they are not held fixed across sizes.

Setting	14M	70M	410M
Parameters	14,067,712	70,426,624	405,334,016
Peak learning rate	10^{-3}	10^{-3}	3×10^{-4}
Minimum learning rate	10^{-4}	10^{-4}	3×10^{-5}
Microbatch (sequences)	32	4	4
Gradient accumulation	32	256	256
Tokens per parameter	106.14	21.20	3.68

D COMPLETE RESULTS AND SUPPORTING DIAGNOSTICS

The following tables report every selected trained endpoint, all 29 paired 14M contrasts and all 15 cross-scale recipe comparisons. They retain dominated settings as well as favorable ones. Each entry summarizes one trained model on complete validation. Sparsity percentages are formed from pooled integer product counts. The endpoint table uses the common A7 architectural-reach denominator from Figure 5, rather than a different denominator for each recipe. This common normalization equals S_{block} .

D.1 ACTIVATION SPARSITY AND SIGNED-DENSITY COVERAGE

The seven frozen-checkpoint evaluations underlying Figure 4 record signed values after the trained nonlinearities, with Q/K measured after RoPE. Each checkpoint contributes 2,658,140,160 FFN elements and 1,594,884,096 attention elements across the complete validation set. Summing integer counts gives the 80:20 weighting of $h : m$ and equal weighting of $q : k : v$. These pooled distributions are element-weighted mixtures, not distributions of layer means.

Exact zeros are stored separately from the nonzero histogram. The native grid spans $[-8, 8]$ with 16,000 bins of nominal width 0.001; the figure combines ten bins at a time and divides by the actual floating-point bin width and the count of all captured elements. Consequently, a curve’s full integral is the recorded nonzero mass inside that grid, not one. Table 9 reports the zero mass and the fractions outside the displayed and native ranges. Underflow and overflow counts are retained rather than folded into edge bins. The displayed limits are $[-2.5, 3]$ for FFN and $[-4, 4]$ for attention. The largest omitted display tail is 6.6719%; the native-grid tail is at most 0.7607%.

The diagnostic reruns use FP32 parameters with FP16 autocast and no additional clipping. Their maximum absolute validation-loss difference from the retained reference is 6.89×10^{-5} , below the specified 5×10^{-4} tolerance; all 108 threshold-consistency checks pass. Small rounding differences in exact-zero counts between evaluation passes do not change the displayed conclusions. Complete per-site and pooled histograms, counts and checkpoint hashes accompany the data release.

Table 6: All 54 trained endpoints. Δ loss uses same-size A0. λ is pressure weight; κ is the trained threshold. Block-only sparsity $\mathcal{S}_{\text{block}} = \mathcal{S}_{\text{model}}/R_{\text{arch}}(\text{A7})$ uses the same A7 reference within each size, including A0. Rounded zero sparsity need not be exactly zero.

Size	Recipe	Parameter	Loss	Δ loss	$\mathcal{S}_{\text{model}}$ (%)	$\mathcal{S}_{\text{block}}$ (%)
14M	A0	—	5.2086	+0.0000	0.000	0.000
14M	A1-H	—	5.2696	+0.0611	2.714	9.061
14M	A1-H-L1	$\lambda = 0.05$	5.2061	-0.0024	3.142	10.489
14M	A1-H-L1	$\lambda = 0.1$	5.1655	-0.0431	3.336	11.139
14M	A1-H-L1	$\lambda = 0.5$	5.1127	-0.0959	3.788	12.645
14M	A1-H-L1	$\lambda = 1$	5.1023	-0.1063	3.949	13.185
14M	A1-H-OL1	$\lambda = 0.05$	5.1981	-0.0105	3.145	10.501
14M	A1-H-OL1	$\lambda = 0.1$	5.1594	-0.0492	3.339	11.146
14M	A1-H-OL1	$\lambda = 0.5$	5.1102	-0.0983	3.768	12.580
14M	A1-H-OL1	$\lambda = 1$	5.1212	-0.0874	3.938	13.149
14M	A4	$\kappa = 0$	5.4705	+0.2619	7.212	24.078
14M	A4	$\kappa = 0.01$	5.4665	+0.2579	7.414	24.752
14M	A4	$\kappa = 0.05$	5.4341	+0.2255	8.206	27.396
14M	A4	$\kappa = 0.1$	5.4197	+0.2111	8.953	29.891
14M	A4	$\kappa = 0.5$	5.6597	+0.4511	10.216	34.106
14M	A4-OL1	$\kappa = 0$	5.4583	+0.2497	7.810	26.075
14M	A4-OL1	$\kappa = 0.01$	5.4583	+0.2497	8.587	28.668
14M	A4-OL1	$\kappa = 0.05$	5.4898	+0.2812	10.456	34.910
14M	A4-OL1	$\kappa = 0.1$	5.5483	+0.3397	11.394	38.041
14M	A4-OL1	$\kappa = 0.5$	6.0380	+0.8294	12.713	42.446
14M	A7	$\kappa = 0$	5.4684	+0.2598	7.218	24.097
14M	A7	$\kappa = 0.01$	5.4588	+0.2502	7.618	25.433
14M	A7	$\kappa = 0.05$	5.4379	+0.2293	9.127	30.471
14M	A7	$\kappa = 0.1$	5.4287	+0.2201	10.425	34.805
14M	A7	$\kappa = 0.5$	5.7029	+0.4943	15.387	51.371
14M	A7-OL1	$\kappa = 0$	5.4802	+0.2716	7.054	23.551
14M	A7-OL1	$\kappa = 0.01$	5.4758	+0.2672	7.723	25.786
14M	A7-OL1	$\kappa = 0.05$	5.4628	+0.2542	9.863	32.929
14M	A7-OL1	$\kappa = 0.1$	5.4295	+0.2209	11.797	39.385
14M	A7-OL1	$\kappa = 0.5$	5.8294	+0.6208	27.483	91.755
70M	A0	—	4.0998	+0.0000	0.000	0.001
70M	A1-H	—	4.2227	+0.1230	10.066	20.366
70M	A4-OL1	$\kappa = 0$	4.8054	+0.7056	25.673	51.944
70M	A4-OL1	$\kappa = 0.01$	4.8229	+0.7231	26.224	53.060
70M	A4-OL1	$\kappa = 0.05$	4.8742	+0.7744	28.490	57.644
70M	A4-OL1	$\kappa = 0.1$	4.9439	+0.8441	30.574	61.861
70M	A4-OL1	$\kappa = 0.5$	5.3895	+1.2897	35.596	72.022
70M	A7-OL1	$\kappa = 0$	4.9412	+0.8414	23.562	47.674
70M	A7-OL1	$\kappa = 0.01$	4.9364	+0.8366	24.226	49.017
70M	A7-OL1	$\kappa = 0.05$	4.9637	+0.8639	26.524	53.666
70M	A7-OL1	$\kappa = 0.1$	4.9501	+0.8503	28.694	58.056
70M	A7-OL1	$\kappa = 0.5$	5.2159	+1.1162	40.602	82.150
410M	A0	—	4.5475	+0.0000	0.011	0.013
410M	A1-H	—	4.6513	+0.1038	21.050	24.127
410M	A4-OL1	$\kappa = 0$	5.6921	+1.1446	38.339	43.944
410M	A4-OL1	$\kappa = 0.01$	5.6914	+1.1439	39.149	44.873
410M	A4-OL1	$\kappa = 0.05$	5.6789	+1.1315	42.125	48.283
410M	A4-OL1	$\kappa = 0.1$	5.6789	+1.1314	45.688	52.367
410M	A4-OL1	$\kappa = 0.5$	5.1910	+0.6435	71.591	82.058
410M	A7-OL1	$\kappa = 0$	5.4263	+0.8788	41.121	47.133
410M	A7-OL1	$\kappa = 0.01$	5.4331	+0.8857	41.478	47.542
410M	A7-OL1	$\kappa = 0.05$	5.4679	+0.9204	44.053	50.494
410M	A7-OL1	$\kappa = 0.1$	5.4907	+0.9433	48.998	56.161
410M	A7-OL1	$\kappa = 0.5$	5.1207	+0.5732	80.616	92.401

Table 7: All 29 Pythia-14M contrasts, treatment minus reference. When both recipes have a swept parameter, its value is matched; A0 and A1-H are fixed controls.

Treatment – reference	Parameter	Δ loss	$\Delta\mathcal{S}_{\text{model}}$ (pp)
A1-H – A0	—	+0.0611	+2.714
A1-H-L1 – A1-H	$\lambda = 0.05$	-0.0635	+0.427
A1-H-L1 – A1-H	$\lambda = 0.1$	-0.1042	+0.622
A1-H-L1 – A1-H	$\lambda = 0.5$	-0.1569	+1.073
A1-H-L1 – A1-H	$\lambda = 1$	-0.1674	+1.235
A1-H-OL1 – A1-H-L1	$\lambda = 0.05$	-0.0081	+0.004
A1-H-OL1 – A1-H-L1	$\lambda = 0.1$	-0.0061	+0.002
A1-H-OL1 – A1-H-L1	$\lambda = 0.5$	-0.0025	-0.020
A1-H-OL1 – A1-H-L1	$\lambda = 1$	+0.0189	-0.011
A4 – A1-H	$\kappa = 0$	+0.2009	+4.498
A4 – A1-H	$\kappa = 0.01$	+0.1969	+4.700
A4 – A1-H	$\kappa = 0.05$	+0.1645	+5.492
A4 – A1-H	$\kappa = 0.1$	+0.1500	+6.239
A4 – A1-H	$\kappa = 0.5$	+0.3900	+7.501
A4-OL1 – A4	$\kappa = 0$	-0.0122	+0.598
A4-OL1 – A4	$\kappa = 0.01$	-0.0082	+1.173
A4-OL1 – A4	$\kappa = 0.05$	+0.0556	+2.251
A4-OL1 – A4	$\kappa = 0.1$	+0.1286	+2.441
A4-OL1 – A4	$\kappa = 0.5$	+0.3783	+2.498
A7 – A4	$\kappa = 0$	-0.0021	+0.006
A7 – A4	$\kappa = 0.01$	-0.0077	+0.204
A7 – A4	$\kappa = 0.05$	+0.0038	+0.921
A7 – A4	$\kappa = 0.1$	+0.0090	+1.472
A7 – A4	$\kappa = 0.5$	+0.0432	+5.171
A7-OL1 – A7	$\kappa = 0$	+0.0118	-0.163
A7-OL1 – A7	$\kappa = 0.01$	+0.0170	+0.106
A7-OL1 – A7	$\kappa = 0.05$	+0.0249	+0.736
A7-OL1 – A7	$\kappa = 0.1$	+0.0008	+1.372
A7-OL1 – A7	$\kappa = 0.5$	+0.1265	+12.096

Table 8: All matched A7-OL1 minus A4-OL1 comparisons. Negative loss changes and positive sparsity changes favor A7-OL1.

Size	κ	Δ loss	$\Delta\mathcal{S}_{\text{model}}$ (pp)
14M	0	+0.0219	-0.756
14M	0.01	+0.0175	-0.863
14M	0.05	-0.0270	-0.593
14M	0.1	-0.1188	+0.402
14M	0.5	-0.2086	+14.769
70M	0	+0.1358	-2.110
70M	0.01	+0.1135	-1.998
70M	0.05	+0.0895	-1.966
70M	0.1	+0.0062	-1.880
70M	0.5	-0.1736	+5.006
410M	0	-0.2658	+2.783
410M	0.01	-0.2582	+2.329
410M	0.05	-0.2111	+1.929
410M	0.1	-0.1881	+3.310
410M	0.5	-0.0703	+9.024

Table 9: Activation sparsity (the probability mass at zero) omitted from the density curves, and mass outside their displayed range (view tail) and the native $[-8, 8]$ histogram (grid tail). Grid tails are a subset of view tails, not an additional mass to add. All percentages divide integer counts by all captured elements. A0 is listed once although it is repeated in each figure row. These 14 groups come from seven checkpoints.

Recipe	κ	Group	Sparsity (%)	View tail (%)	Grid tail (%)
A0	—	FFN	< 0.01	0.1113	0.0000
A0	—	Attention	< 0.01	2.3929	0.4632
A4-OL1	0	FFN	67.22	0.0003	0.0000
A4-OL1	0	Attention	< 0.01	5.1717	0.6162
A4-OL1	0.05	FFN	86.71	0.0003	0.0000
A4-OL1	0.05	Attention	< 0.01	5.6601	0.7607
A4-OL1	0.5	FFN	99.31	0.0001	0.0000
A4-OL1	0.5	Attention	0.22	0.0003	0.0000
A7-OL1	0	FFN	64.52	0.0063	0.0000
A7-OL1	0	Attention	< 0.01	5.4657	0.4648
A7-OL1	0.05	FFN	75.29	0.0106	0.0000
A7-OL1	0.05	Attention	7.26	6.6719	0.6932
A7-OL1	0.5	FFN	93.64	0.0037	0.0000
A7-OL1	0.5	Attention	95.60	0.2396	0.0009

D.2 PROJECTION-INPUT VECTORS IN THE BF16 KERNEL DIAGNOSTIC

An untimed diagnostic pass retained the distribution of nonzero counts per token row at each site and layer for all 30 K050 checkpoints. These are BF16 kernel operands on all 338 validation blocks, separate from the FP16 histogram reruns above. Every layer contributes 692,224 rows; h has 512 coordinates per row and z has 128. Table 10 reports the two $\kappa = 0.5$ pressured recipes. Summing the row histograms recovers the captured sitewise zero counts. Pooling rows across layers gives the all-zero-vector rates reported in Section 4.3. The records do not retain token identities or context perturbations, and zero projection inputs can still produce bias outputs. They therefore do not establish whether a branch has lost its function.

Table 10: Entirely zero projection-input rows and mean nonzero coordinates per row (NNZ), separately at h and z , for the 14M $\kappa = 0.5$ endpoints. Layer indices start at zero. All entries use actual BF16 operands in the untimed K050 diagnostic; they are not the pooled FP16 h, m histograms.

Recipe	Layer	$h = 0$ rows (%)	Mean NNZ h	$z = 0$ rows (%)	Mean NNZ z
A4-OL1	0	12.45	1.729	90.34	0.116
A4-OL1	1	99.63	0.004	100.00	0.000
A4-OL1	2	100.00	0.000	100.00	0.000
A4-OL1	3	99.91	0.001	100.00	0.000
A4-OL1	4	99.59	0.011	92.59	0.074
A4-OL1	5	90.77	0.114	100.00	0.000
A7-OL1	0	22.60	2.022	69.09	0.405
A7-OL1	1	85.01	0.182	96.38	0.045
A7-OL1	2	75.10	0.296	95.72	0.047
A7-OL1	3	78.99	0.263	87.77	0.123
A7-OL1	4	62.28	0.526	98.24	0.018
A7-OL1	5	54.77	0.536	100.00	0.000

D.3 OPERATION CONTRIBUTIONS TO MODEL-WIDE SPARSITY

An operation’s contribution to S_{model} is its zero-product count divided by the full model product count. Summing these contributions recovers the model-wide fraction; summing local activation percentages would not. Table 11 gives this decomposition at $\kappa = 0.5$. At 14M, extending from A4-OL1 to A7-OL1 adds attention opportunity while reducing the contribution from some projections. At larger sizes, projection work carries more weight in the workload, so an equal change in activation

sparsity need not yield an equal change in model-wide sparsity. Figure 7 displays the operation-level view at the same threshold and all three sizes.

Table 11: Contributions to $100S_{\text{model}}$ in percentage points at $\kappa = 0.5$. All six operations share the same model denominator within each size. The dense LM head contributes to that denominator but receives no zero credit. Differences in rounded entries can affect the displayed sum.

Operation	14M		70M		410M	
	A4-OL1	A7-OL1	A4-OL1	A7-OL1	A4-OL1	A7-OL1
QKV	3.173	2.430	8.820	6.231	17.563	16.992
<i>QK</i>	0.000	8.324	0.000	5.052	0.000	5.206
<i>PV</i>	0.057	8.449	0.011	5.371	0.008	5.858
Attention output	1.069	1.069	3.087	3.084	6.186	6.187
FFN up	4.139	2.939	11.329	8.526	22.938	21.503
FFN down	4.275	4.272	12.350	12.338	24.895	24.869
Total	12.713	27.483	35.596	40.602	71.591	80.616

Operation contributions at $\kappa = .5$

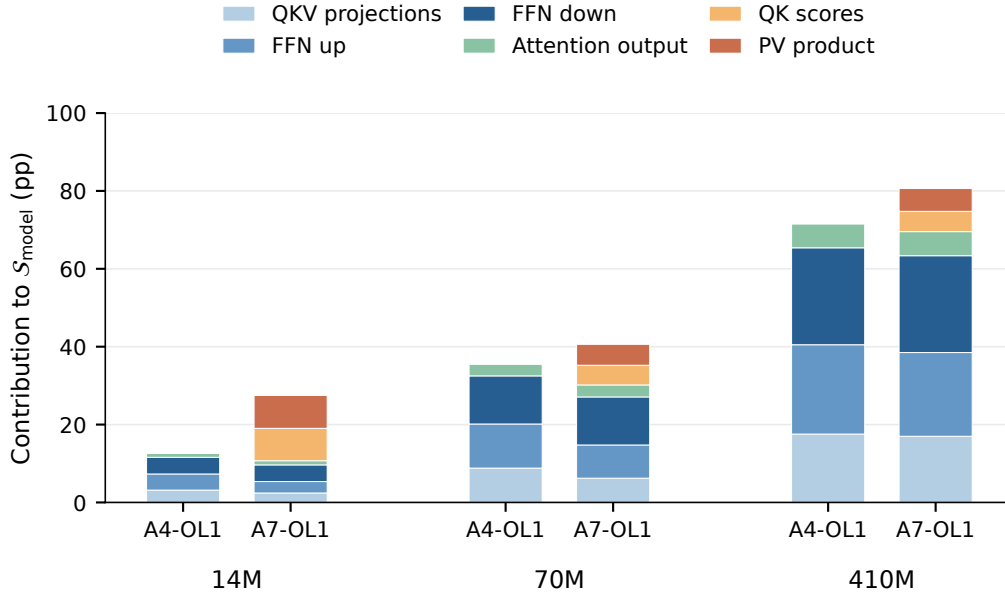


Figure 7: Operation accounting for A4-OL1 and A7-OL1 at $\kappa = 0.5$ across 14M, 70M and 410M. Zero-operand products are counted over causally valid positions and pooled across all 338 validation blocks. Each contribution uses the full model denominator, including the dense LM head without zero credit. At 14M, attention contributes more under A7-OL1, outweighing smaller contributions from several projections. The bars show logical multiplication opportunities, not runtime savings.

D.4 COMPLETE POST-HOC CLIPPING TRAJECTORIES

The complete release contains 540 evaluations: ten clipping targets for each of the 54 trained checkpoints. It joins 350 new evaluations with 190 retained evaluations matched by checkpoint content hash. Calibration uses the first ten complete training blocks in source order, separately for each layer’s a, m, h, z activations. Evaluation zeros values satisfying $|x| \leq t$ after the trained nonlinearities, without changing weights or A7’s internal attention nonlinearities. The target p controls a

calibration quantile; it is not the achieved model-wide sparsity. In particular, an existing zero mass can make several targets identical.

Figures 8–10 retain the full loss range and every target, including dominated points and ties. Each path begins at its actually measured $p = 0$ evaluation. Small numerical differences from the canonical training endpoint can change strict nondominance without implying a substantive improvement. Across all checkpoints, the absolute $p = 0$ discrepancy is at most 1.81×10^{-4} in loss and 0.000908 sparsity percentage points. Checking dominance beyond the observed same-size drift on both axes still finds a trained endpoint with lower loss and higher sparsity than each A0 clipping evaluation at $p \geq 0.5$, for every size. This is a numerical sensitivity check, not a confidence interval or an estimate of training variability. Numerical frontier memberships are stored separately from the lines connecting evaluated settings. All evaluations use complete validation with FP32 parameters, FP16 autocast, eager attention, batch one and $T = 2,048$.

14M training and post-hoc clipping

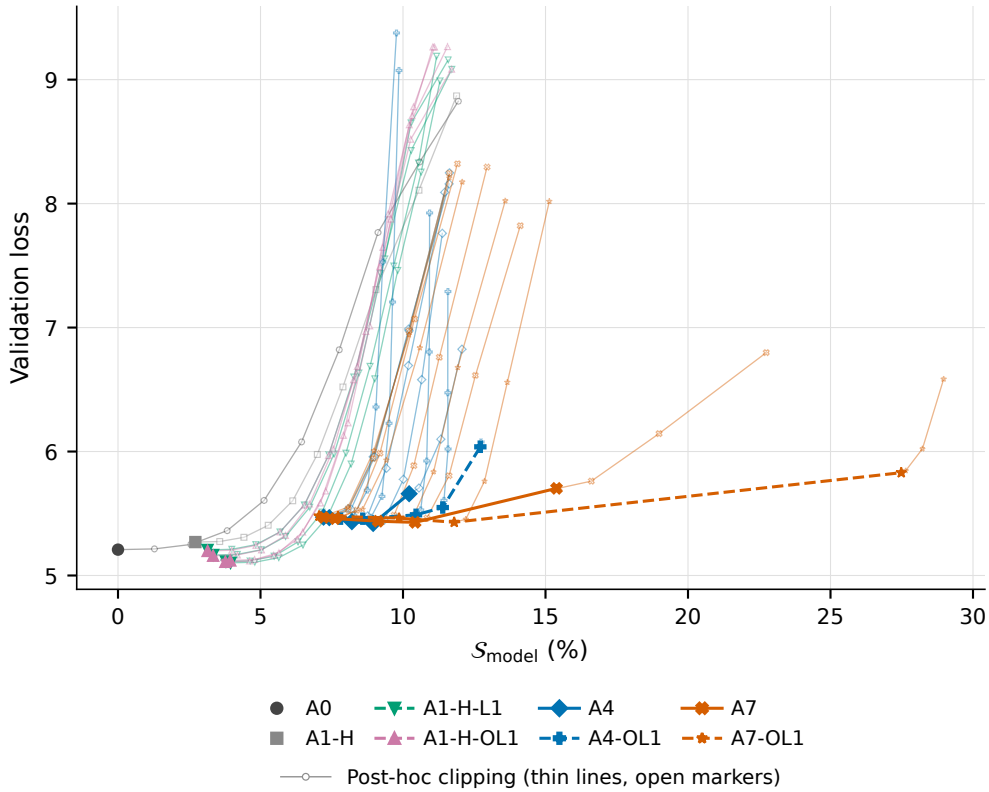


Figure 8: Complete 14M training and post-hoc clipping results: 30 trained endpoints and 300 clipping evaluations. Large filled markers denote trained endpoints; bold lines connect their settings, solid for unpressured recipes and dashed for L1/OL1. Each faint open-marker path follows one fixed checkpoint through $p = 0, 0.1, \dots, 0.9$. Colors and symbols identify its source recipe. The full loss range is shown. Connections order evaluated settings and do not interpolate attainable models or fitted Pareto envelopes.

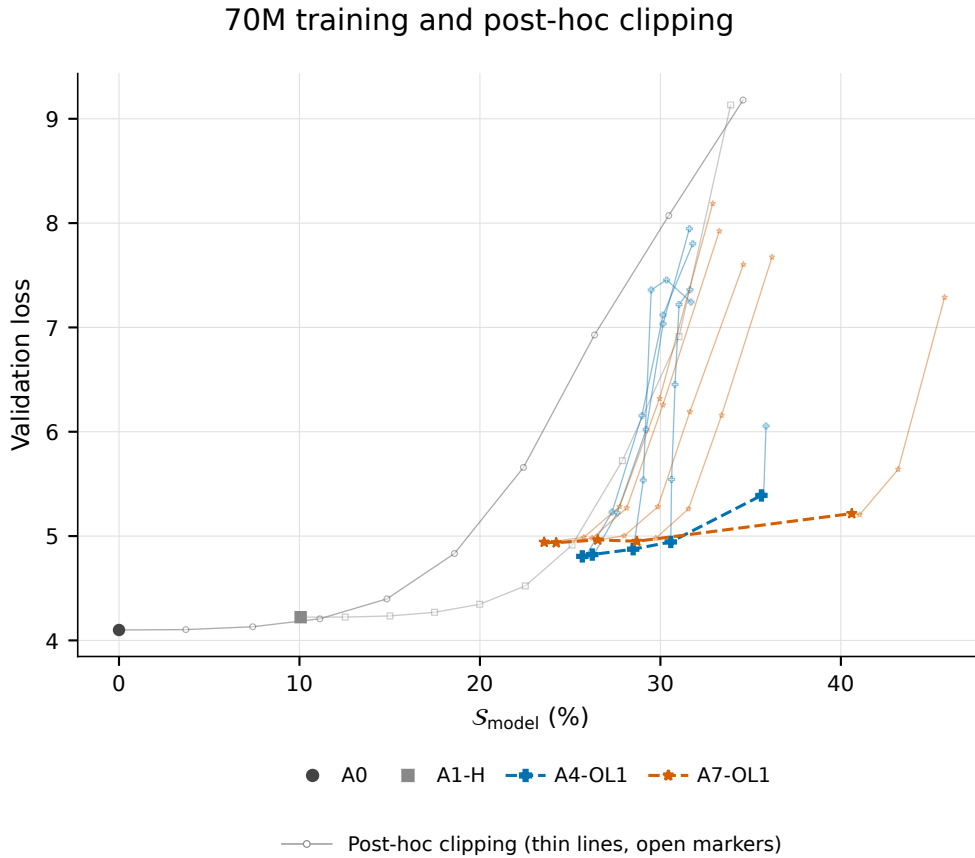


Figure 9: Complete 70M training and post-hoc clipping results: 12 trained endpoints and 120 clipping evaluations. Large filled markers and bold lines identify the trained recipes; faint open-marker paths follow each checkpoint through ten increasing clipping targets. All points, including overlaps and high-loss settings, are retained. The evaluation and line conventions match Figure 8.

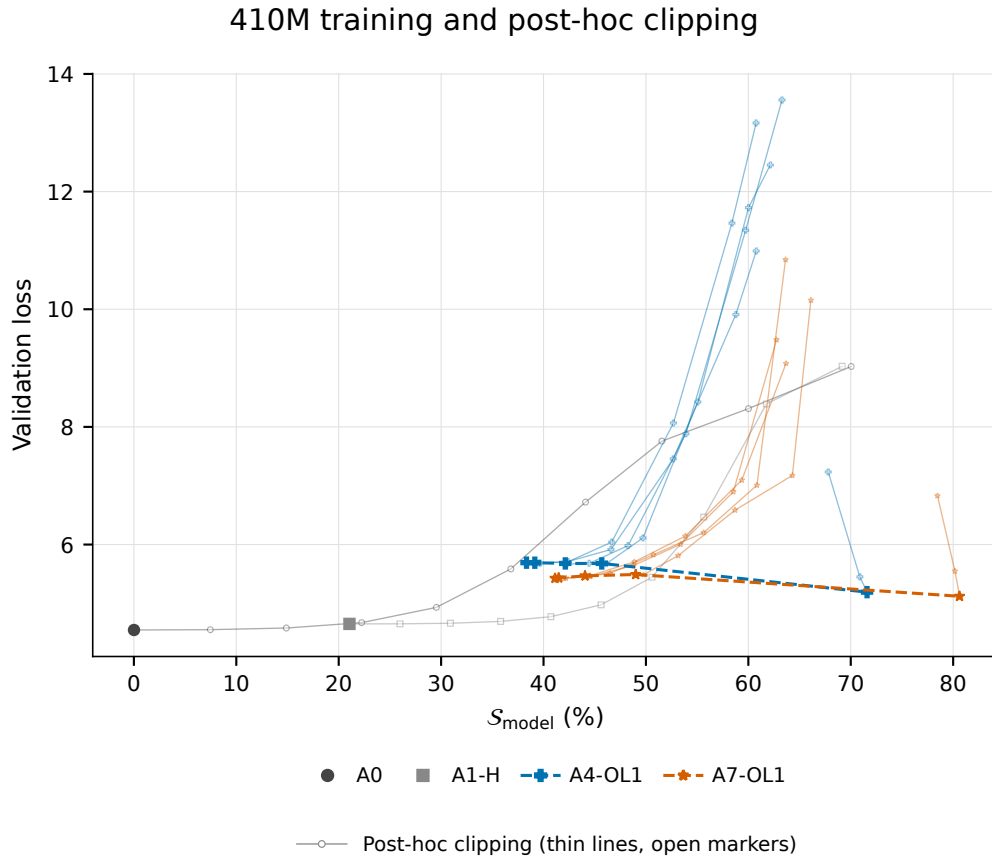


Figure 10: Complete 410M training and post-hoc clipping results: 12 trained endpoints and 120 clipping evaluations. Large filled markers and bold lines identify the trained recipes; faint open-marker paths follow each checkpoint through ten increasing clipping targets. All points, including overlaps and high-loss settings, are retained. The evaluation and line conventions match Figure 8.

D.5 KERNEL QUALIFICATION AND ABLATIONS

The unchanged Sakana/TwELL control achieves $1.3001\times$ on its supported SparseLM-0.5B/H100 workload. Its raw projection primitive at the Pythia-14M width $N = 128$ instead gives relative L2 error 0.69055, exceeding the 0.02 bound; that test stops before full-model Pythia timing. This is a compatibility test, not an unchanged upstream speedup comparison on our model. The later P0 adapter changes signed-value packing, shape tails and numerical handling for Pythia. It retains dense attention and uses native projection fallback on A0. Table 12 reports this adapted baseline.

Historical proposals are replayed on four representative checkpoints; the plotted incumbent uses fixed A7-OL1, $\kappa = 0.5$. Final implementation and ablation summaries retain the 30-checkpoint manuscript cohort from the historical 35-checkpoint sweep. Five A4 checkpoints with pressure only at h remain outside the main cohort because they implement a distinct objective. Their original configurations, inherited capture code, initialization, schedule and checkpoint identities were reconciled against the corrected four-site runs. Across the five matched thresholds, four-site pressure raises diagnostic loss by 0.2425–0.3196 relative to h -only pressure. Its sparsity change is -0.599 pp at zero threshold, $+0.001$ pp at 0.01, and $+1.421$ – $+2.486$ pp at larger thresholds. Adding pressure sites changes each old site–layer coefficient from $1/6$ to $1/24$, so this historical comparison does not hold per-site pressure fixed. Search progress and the final sweep are separate measurements, explaining the small difference between their $1.7830\times$ and $1.7832\times$ maxima.

Matched retrospective search and native-relative association

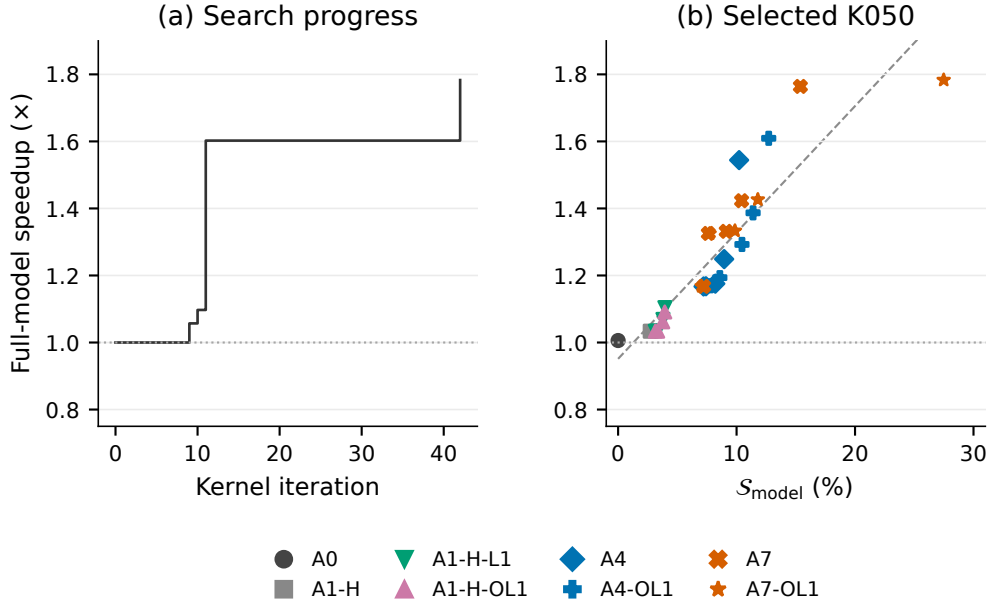


Figure 11: Retained development history and descriptive native-relative association. (a) Best qualified incumbent on the fixed A7-OL1, $\kappa = 0.5$ search checkpoint: $1.6024\times$ by iteration 11 and $1.7830\times$ at iteration 42. The later attention-dense ablation is not substituted into this history. (b) Selected K050 on all 30 canonical checkpoints, with an OLS fit and intercept ($R^2 = 0.8167$). Sparsity uses FP16 counts and timing uses BF16; the axes have restricted ranges. Both panels use the matched RTX5090, batch-one, full-sequence native reference specified in the text.

The coding-agent configuration records *gpt-6-astra* as the default model, not the model served for every request. This is one human-guided development trajectory and does not compare agent-assisted with human-only kernel development. Kernel measurements use Python 3.12.3, PyTorch 2.11.0+cu128, Transformers 5.12.1, CUDA 12.8 and nvcc 12.8.93. The supplementary protocol records dependency and frozen-source hashes.

Every candidate must produce finite outputs, with each logit satisfying $|y - \hat{y}| \leq 0.25 + 0.02|\hat{y}|$, per-block relative output L2 error at most 0.02, and absolute pooled loss difference at most 0.001. Here $\hat{y}$ is the unmodified native eager reference. All three fresh processes must pass on all 338 validation blocks. Qualification is bounded numerical agreement on these inputs, not bitwise equivalence. Failed candidates are excluded from qualified speedup summaries. P0, an early projection adapter derived from the released kernels of Cetin et al. (2026), qualifies on only four checkpoints. Its average is not comparable to a 30-checkpoint mean or an unchanged upstream benchmark.

Timing uses the same checkpoint, inputs and CUDA-graph scaffold for native PyTorch/SDPA and each candidate. For each checkpoint, we take the geometric mean of 1,344 paired host-latency ratios (native divided by candidate): 64 inputs, seven randomized paired passes, and three fresh processes. Compilation, static weight preparation, graph capture and equal input staging are excluded. Preprocessing that depends on activations, embeddings, every block, final normalization and full vocabulary logits are included. There is no activation cache. Cohort summaries give each checkpoint equal weight.

K050 combines normalization and RoPE fusion, zero-fragment tensor-core bypass and sparse-row projection paths, using CUDA, CUTLASS, FlashAttention and PyTorch components. K049 is its searched predecessor; K050 adds fusion of the two input LayerNorms while preserving their separate affine parameters and thresholds. The all-skips-off control retains fusion while disabling both attention and projection skips; the attention-dense control retains projection skips. Their ratios are formed from separately measured, native-normalized timings. K050’s sparse paths improve 14 of 30 checkpoints relative to all-skips-off, with geometric mean ratio $1.0432\times$. Disabling attention skipping improves all 30: at the search checkpoint, instrumentation confirms that skipping bypasses 58.0% of eligible QK and 67.2% of PV tensor-core matrix multiply-accumulate (MMA) instructions, yet its overhead exceeds the saved time. Scalar zero-product fractions and skipped MMA fractions are different estimands.

The attention path tests whether either warp-distributed operand fragment contains only zeros, and issues an MMA only when both fragments contain a nonzero. Partial sparsity can therefore leave an entire instruction active. Zero detection, warp votes and branches recur inside the kernel; causal masking and softmax still execute. A zero QK score does not imply a zero softmax probability. The measurements do not identify a profitable joint-zero pattern or break-even threshold. They show that the skipped work does not repay these costs for the measured 14M shapes and hardware.

Table 12: Numerically qualified full-model speedups against native SDPA CUDA graphs. GM is the equal-checkpoint geometric mean; the range is across qualified checkpoints, not a confidence interval. All conditions use one RTX5090, BF16, batch one, 2,048 tokens and full vocabulary logits. P0’s four-checkpoint subset is shown only to document its limited qualification.

Implementation	Qualified	GM speedup	Range
K049	30/30	1.1141	0.9550–1.5284
K050	30/30	1.2340	1.0059–1.7832
K050, attention dense	30/30	1.2506	1.0185–1.8022
K050, all skips off	30/30	1.1829	1.0503–1.3628
P0 (qualified subset only)	4/30	0.8910	0.8180–1.0012

D.5.1 ABSOLUTE LATENCIES AND ASSOCIATION CHECKS

Table 13 retains actual host latencies: for each implementation and checkpoint, the geometric mean of the three process medians, each over 448 timed sequences. The native column uses K050’s paired native measurements; the ablation columns have their own native-paired processes. A ratio of these median summaries need not equal the reported geometric mean of paired ratios. The last column instead divides K050’s native-relative speedup by all-skips-off’s native-relative speedup. BF16 validation losses accompany these timings; canonical training tables retain their original FP16 loss/count pass.

The association checks in Table 14 fit an unweighted checkpoint-level line with an intercept. Removing the searched checkpoint or a recipe family tests descriptive sensitivity; it does not create a held-out prediction study. Within-family fits use only four or five points, and their larger R^2 values

Table 13: Matched 14M host latency summaries in milliseconds, BF16 validation loss and incremental sparse-path factor. Dose is λ for local pressure and κ for A4/A7. All four implementations qualify on every row. These are unmodified trained checkpoints; clipped endpoints have no corresponding latency measurements. The sparse factor is defined in the text, not computed by dividing the displayed rounded latency columns.

ID	Recipe	Dose	BF16 loss	Native	Skips off	K050	Attn. dense	Factor
c01	A0	—	5.2100	0.6555	0.6213	0.6516	0.6437	0.9535
c02	A1-H	—	5.2712	0.6539	0.6224	0.6328	0.6242	0.9835
c03	A1-H-L1	0.05	5.2082	0.6541	0.6221	0.6329	0.6237	0.9828
c04	A1-H-L1	0.1	5.1668	0.6539	0.6226	0.6324	0.6240	0.9843
c05	A1-H-L1	0.5	5.1136	0.6541	0.6220	0.6120	0.6044	1.0162
c06	A1-H-L1	1	5.1040	0.6543	0.6231	0.5922	0.5838	1.0512
c07	A1-H-OL1	0.05	5.1991	0.6540	0.6225	0.6326	0.6254	0.9843
c08	A1-H-OL1	0.1	5.1609	0.6540	0.6223	0.6331	0.6247	0.9833
c09	A1-H-OL1	0.5	5.1114	0.6542	0.6230	0.6159	0.6082	1.0106
c10	A1-H-OL1	1	5.1228	0.6540	0.6219	0.5989	0.5909	1.0372
c11	A4	0	5.4727	0.7399	0.6234	0.6338	0.6250	0.9820
c12	A4	0.01	5.4694	0.7402	0.6234	0.6339	0.6251	0.9839
c13	A4	0.05	5.4364	0.7400	0.6233	0.6295	0.6213	0.9898
c14	A4	0.1	5.4218	0.7399	0.6230	0.5917	0.5836	1.0511
c15	A4	0.5	5.6623	0.7394	0.6242	0.4790	0.4713	1.3022
c16	A4-OL1	0	5.4598	0.7402	0.6229	0.6331	0.6247	0.9842
c17	A4-OL1	0.01	5.4602	0.7401	0.6239	0.6191	0.6108	1.0054
c18	A4-OL1	0.05	5.4922	0.7401	0.6236	0.5706	0.5628	1.0885
c19	A4-OL1	0.1	5.5504	0.7399	0.6233	0.5325	0.5249	1.1674
c20	A4-OL1	0.5	6.0403	0.7394	0.6227	0.4591	0.4511	1.3542
c21	A7	0	5.4704	0.7400	0.6227	0.6339	0.6260	0.9824
c22	A7	0.01	5.4611	0.8460	0.6212	0.6382	0.6306	0.9734
c23	A7	0.05	5.4405	0.8458	0.6208	0.6352	0.6267	0.9773
c24	A7	0.1	5.4309	0.8453	0.6209	0.5934	0.5858	1.0443
c25	A7	0.5	5.7076	0.8449	0.6211	0.4789	0.4714	1.2955
c26	A7-OL1	0	5.4824	0.7400	0.6231	0.6337	0.6255	0.9831
c27	A7-OL1	0.01	5.4778	0.8456	0.6213	0.6384	0.6308	0.9725
c28	A7-OL1	0.05	5.4651	0.8459	0.6214	0.6346	0.6274	0.9782
c29	A7-OL1	0.1	5.4315	0.8459	0.6210	0.5920	0.5847	1.0473
c30	A7-OL1	0.5	5.8313	0.8441	0.6210	0.4731	0.4679	1.3112

do not remove the differences among recipes. All fits compare canonical FP16 $\mathcal{S}_{\text{model}}$ with BF16 timing. The retained BF16 scalar counter excludes probability-underflow zeros and is only a lower bound, so it cannot replace the canonical counter to resolve that precision mismatch.

Table 14: Descriptive association sensitivity. Each column regresses its timing ratio on canonical FP16 model-wide sparsity with an intercept. Native relative uses K050/native; sparse factor uses the incremental definition above. Subsets are retrospective and overlapping, with no independent training seeds.

Checkpoint subset	n	Native-relative R^2	Sparse-factor R^2
All checkpoints	30	0.8167	0.5037
Within A1-H-L1	4	0.8944	0.8908
Within A1-H-OL1	4	0.8888	0.8921
Within A4	5	0.8353	0.8346
Within A4-OL1	5	0.9087	0.9095
Within A7	5	0.9528	0.9356
Within A7-OL1	5	0.9403	0.9825
Without A0	29	0.8104	0.4943
Without A1-H	29	0.8101	0.4989
Without A1-H-L1	26	0.7937	0.5077
Without A1-H-OL1	26	0.7929	0.5007
Without A4	25	0.8497	0.5685
Without A4-OL1	25	0.8319	0.5322
Without A7	25	0.8401	0.4977
Without A7-OL1	25	0.8520	0.4910
Without search checkpoint	29	0.8521	0.4080

D.6 EVIDENCE AND DATA ACCOMPANYING THE DRAFT

The supplementary directory `supplementary-data/` contains all 54 training records with integer operation counts, all 540 clipping evaluations and frontier memberships, seven signed histograms with zero and tail counts, figure coordinates, and the 30-checkpoint runtime reduction. The included README defines fields and units. Copy hashes are recorded in `SOURCES.json`; `protocol.json` gives data preparation, training settings and implementation identities. Paths refer to the accompanying repository; the supplementary directory is not a standalone checkpoint or kernel release. The `revision-audit/` subdirectory adds the retrospective quality-budget reduction, absolute runtime summaries and sensitivity checks, per-layer projection-input row statistics, and historical-pressure identity audit. Raw identifiers remain unchanged: $\mathcal{S}_{\text{model}}$ corresponds to `R_model`, $\mathcal{S}_{\text{block}}$ to `R_block`, and architectural reach R_{arch} to the existing `R_model_max` fields.

The accompanying repository includes the figure and table scripts. The Analysis 019 README gives commands for regenerating the revised visuals from the retained numerical records, separate from commands that require the original raw run artifacts. The draft README gives the LaTeX build commands. Checkpoint weights and runtime compilation dependencies are not bundled in the supplementary measurement directory; these commands do not claim to reproduce training or GPU measurements from that directory alone.